

```

1 using CamBam.Geom;
2 using CamBam.CAD;
3 using System.Collections.Generic;
4
5 public static class SurfaceUtils
6 {
7     public static Surface ConvertTrianglesToSurface(List<Triangle> triangles)
8     {
9         Point3FArray points = new Point3FArray();
10        Dictionary<Point3F, int> pointIndex = new Dictionary<Point3F, int>();
11        List<TriangleFace> faces = new List<TriangleFace>();
12
13        for (int i = 0; i < triangles.Count; i++)
14        {
15            Triangle t = triangles[i];
16
17            int ia = AddPoint(t.A, points, pointIndex);
18            int ib = AddPoint(t.B, points, pointIndex);
19            int ic = AddPoint(t.C, points, pointIndex);
20
21            faces.Add(new TriangleFace(ia, ib, ic));
22        }
23
24        Surface surface = new Surface();
25        surface.Points = points;
26        surface.Faces = faces.ToArray();
27        return surface;
28    }
29
30    private static int AddPoint(Point3F p, Point3FArray points, Dictionary<Point3F,
31    int> indexMap)
32    {
33        int index;
34        if (!indexMap.TryGetValue(p, out index))
35        {
36            index = points.Count;
37            points.Add(p);
38            indexMap[p] = index;
39        }
40        return index;
41    }
42 using CamBam.CAD;
43 using CamBam.Geom;
44 using System.Collections.Generic;
45
46 namespace MorphMuse.Services
47 {
48     public class FanCapGenerator
49     {
50         static public void FindConvergenceCentersAndCurves(
51             Polyline closedBase,
52             double minOffset,
53             double maxOffset,
54             float tolerance,
55             out List<Point3F> centers,
56             out List<Polyline> finalCurves)
57         {
58             centers = new List<Point3F>();
59             finalCurves = new List<Polyline>();
60
61             double low = minOffset;
62             double high = maxOffset;
63
64             while ((high - low) > 0.01)
65             {
66                 double mid = (low + high) / 2.0;
67                 Polyline[] offsets = closedBase.CreateOffsetPolyline(-mid, tolerance);
68
69                 if (offsets == null || offsets.Length == 0)
70                 {
71                     high = mid;
72                 }
73             }
74         }
75     }
76 }

```

```

73         else
74         {
75             low = mid;
76             centers.Clear();
77             finalCurves.Clear();
78
79             for (int i = 0; i < offsets.Length; i++)
80             {
81                 Polyline poly = offsets[i];
82                 if (poly.Closed && poly.Points.Count >= 3)
83                 {
84                     finalCurves.Add(poly);
85                     centers.Add(poly.GetCentroid()); // Usa API do CamBam
86                 }
87             }
88         }
89     }
90 }
91 static public List<TriangleFace> GenerateFanCapForMultiple(
92     List<Polyline> curves,
93     List<Point3F> centers,
94     Point3FArray points,
95     Dictionary<Point3F, int> pointIndex)
96 {
97     List<TriangleFace> faces = new List<TriangleFace>();
98     int count = System.Math.Min(curves.Count, centers.Count);
99
100     for (int i = 0; i < count; i++)
101     {
102         Polyline curve = curves[i];
103         Point3F center = centers[i];
104
105         if (!curve.Closed || curve.Points.Count < 3)
106             continue;
107
108         int ic = Geometry3F.AddPoint(center, points, pointIndex);
109
110         for (int j = 0; j < curve.Points.Count - 1; j++)
111         {
112             int ia = Geometry3F.AddPoint(curve.Points[j].Point, points,
113 pointIndex);
114             int ib = Geometry3F.AddPoint(curve.Points[j + 1].Point, points,
115 pointIndex);
116             faces.Add(new TriangleFace(ic, ia, ib));
117
118             int iaLast = Geometry3F.AddPoint(curve.Points[curve.Points.Count -
119 1].Point, points, pointIndex);
120             int ibFirst = Geometry3F.AddPoint(curve.Points[0].Point, points,
121 pointIndex);
122             faces.Add(new TriangleFace(ic, iaLast, ibFirst));
123         }
124     }
125     return faces;
126 }
127
128 public static Point3F CalculateCentroid(Polyline poly)
129 {
130     float cx = 0f;
131     float cy = 0f;
132     float cz = 0f;
133
134     for (int i = 0; i < poly.Points.Count; i++)
135     {
136         Point3F pt = poly.Points[i].Point;
137         cx += (float)pt.X;
138         cy += (float)pt.Y;
139         cz += (float)pt.Z;
140     }
141
142     int count = poly.Points.Count;
143     return new Point3F(cx / count, cy / count, cz / count);
144 }

```

```

142     }
143 }
144 using CamBam.CAD;
145 using CamBam.Geom;
146 using System.Collections.Generic;
147
148 namespace MorphMuse.Services
149 {
150     public static class OffsetUtils
151     {
152         public static List<List<Polyline>> GenerateOffsetLayersRecursive(
153             List<Polyline> basePolylines,
154             float offsetStep,
155             int maxLayers,
156             float tolerance)
157         {
158             var result = new List<List<Polyline>>();
159             GenerateOffsetLayersRecursiveInternal(basePolylines, offsetStep,
160 maxLayers, tolerance, result, 0);
161             return result;
162         }
163
164         private static void GenerateOffsetLayersRecursiveInternal(
165             List<Polyline> currentPolylines,
166             float offsetStep,
167             int maxLayers,
168             float tolerance,
169             List<List<Polyline>> result,
170             int currentLayer)
171         {
172             if (currentLayer >= maxLayers || currentPolylines == null ||
173 currentPolylines.Count == 0)
174                 return;
175
176             result.Add(currentPolylines);
177
178             var nextLayerPolylines = new List<Polyline>();
179             foreach (var poly in currentPolylines)
180             {
181                 // Offset each polyline; may return multiple polylines due to
182                 singularities
183                 Polyline[] offsets = poly.CreateOffsetPolyline(offsetStep, tolerance);
184                 if (offsets != null && offsets.Length > 0)
185                 {
186                     nextLayerPolylines.AddRange(offsets);
187                 }
188             }
189
190             // Recursively process the next layer
191             GenerateOffsetLayersRecursiveInternal(nextLayerPolylines, offsetStep,
192 maxLayers, tolerance, result, currentLayer + 1);
193         }
194     }
195 }
196
197 using CamBam.CAD;
198 using CamBam.Geom;
199 using System;
200 using System.Collections.Generic;
201 using MorphMuse.Services;
202
203 namespace MorphMuse
204 {
205     internal class MeshBuilder
206     {
207         {
208             public static List<List<Point3F>> GenerateSampledCurvesFromGeratriz(
209                 List<List<Polyline>> orderedContours,
210                 List<Point3F> simplifiedGeratriz,
211                 double baseDensity = 0.5,
212                 double minStep = 0.1)
213             {
214                 var sampledCurves = new List<List<Point3F>>();
215
216                 for (int i = 0; i < orderedContours.Count; i++)

```

```

211         {
212             var curves = orderedContours[i];
213             double step = baseDensity;
214
215             if (i < simplifiedGeratriz.Count - 1)
216             {
217                 // Corrected: Use Geometry3F.FromPoints instead of subtraction
218                 double spacing =
Geometry3F.Length(Geometry3F.FromPoints(simplifiedGeratriz[i], simplifiedGeratriz[i +
1])));
219                 step = Math.Max(minStep, spacing * baseDensity);
220             }
221
222             foreach (var curve in curves)
223             {
224                 Vector3F delta = Geometry3F.FromPoints(simplifiedGeratriz[i],
simplifiedGeratriz[i + 1]);
225                 double spacing = Geometry3F.Length(delta);
226                 double stp = Math.Max(minStep, spacing * baseDensity);
227
228                 PointList rawPoints =
PointListUtils.CreatePointlistFromPolylineStep(curve, stp);
229                 List<Point3F> sampled = ConvertPointList(rawPoints);
230                 sampledCurves.Add(sampled);
231             }
232         }
233
234         return sampledCurves;
235     }
236
237     public static List<Point3F> ConvertPointList(PointList pointList)
238     {
239         var result = new List<Point3F>();
240         for (int i = 0; i < pointList.Points.Count; i++)
241         {
242             result.Add(pointList.Points[i]);
243         }
244         return result;
245     }
246 }
247
248 using CamBam.CAD;
249 using CamBam.Geom;
250 using CamBam.UI;
251 using MorphMuse.Services;
252 using System.Collections.Generic;
253 using System.Drawing;
254
255 public class MorphMuseController
256 {
257     private readonly CamBamUI _ui;
258
259     public MorphMuseController(CamBamUI ui)
260     {
261         _ui = ui;
262     }
263
264     public void Execute()
265     {
266         // Attempt to retrieve selected polylines from the current CAD view
267         if (!PolylineManager.TryCreateFromSelection(out PolylineManager manager))
268             return;
269
270         // Process the open polyline (generatrix) with smoothing parameters
271         var processor = new OpenPolylineProcessor(manager.OpenPoly, 0.01, 0.01);
272
273         // Order closed polylines (contours) based on their relation to the generatrix
274         var orderedContours = LayerGenerator.GenerateContoursByGeratrizOrder(
275             manager.ClosedPoly,
276             processor.SimplifiedPoints
277         );
278
279         // Sample points along the contours with controlled density

```

```

280     var sampledCurves = CurveSampler.GenerateSampledPointsFromContours(
281         orderedContours,
282         processor.SimplifiedPoints,
283         0.05,
284         0.05
285     );
286
287     // Simplify each sampled curve to reduce geometric complexity
288     var simplifiedCurves = SimplifyAll(sampledCurves);
289
290     // Abort if there are not enough curves to build a surface
291     if (simplifiedCurves.Count < 2)
292         return;
293
294     // Save the currently active layer to restore it later
295     string originalLayerName = _ui.ActiveView.CADFile.ActiveLayerName;
296
297     // Create a new layer to store the final mesh
298     string layerName = CreateUniqueLayer("Mesh");
299     Layer layer = _ui.ActiveView.CADFile.Layers[layerName];
300     layer.Color = Color.DeepSkyBlue;
301
302     // Structures to accumulate all vertices and faces of the final surface
303     Point3FArray allPoints = new Point3FArray();
304     Dictionary<Point3F, int> pointIndex = new Dictionary<Point3F, int>();
305     List<TriangleFace> allFaces = new List<TriangleFace>();
306
307     // Generate surfaces between each pair of consecutive curves
308     for (int i = 0; i < simplifiedCurves.Count - 1; i++)
309     {
310         var lower = simplifiedCurves[i];
311         var upper = simplifiedCurves[i + 1];
312
313         Surface partialSurface =
314         SurfaceBuilderCopilot.BuildSurfaceBetweenCurves(lower, upper);
315
316         // Reindex the vertices of the partial surface into the final mesh
317         for (int f = 0; f < partialSurface.Faces.Length; f++)
318         {
319             TriangleFace face = partialSurface.Faces[f];
320
321             Point3F pa = partialSurface.Points[face.A];
322             Point3F pb = partialSurface.Points[face.B];
323             Point3F pc = partialSurface.Points[face.C];
324
325             int ia = AddPoint(pa, allPoints, pointIndex);
326             int ib = AddPoint(pb, allPoints, pointIndex);
327             int ic = AddPoint(pc, allPoints, pointIndex);
328
329             allFaces.Add(new TriangleFace(ia, ib, ic));
330         }
331     }
332
333     // Create the final consolidated surface
334     Surface finalSurface = new Surface
335     {
336         Points = allPoints,
337         Faces = allFaces.ToArray()
338     };
339
340     // Add the surface to the newly created layer
341     _ui.ActiveView.CADFile.Add(finalSurface);
342
343     // Restore the previously active layer
344     _ui.ActiveView.CADFile.SetActiveLayer(originalLayerName);
345
346     // Adjust the view to fit the new geometry and refresh the display
347     _ui.ActiveView.ZoomToFit();
348     _ui.ActiveView.RefreshView();
349 }
350
351 public void GenerateOffsetSurfaces(Polyline closedBase, double offsetStep, float
tolerance)

```

```

351     {
352         var cadFile = _ui.ActiveView.CADFile;
353
354         // Cria layer único para a malha completa
355         string layerName = CreateUniqueLayer("MalhaOffset_");
356         Layer layer = cadFile.Layers[layerName];
357         layer.Color = Color.DeepSkyBlue;
358
359         // Estruturas acumuladoras
360         Point3FArray allPoints = new Point3FArray();
361         Dictionary<Point3F, int> pointIndex = new Dictionary<Point3F, int>();
362         List<TriangleFace> allFaces = new List<TriangleFace>();
363
364         // Inicializa com a curva base
365         List<Polyline> currentLayer = new List<Polyline> { closedBase };
366
367         while (currentLayer.Count > 0)
368         {
369             List<Polyline> nextLayer = new List<Polyline>();
370
371             foreach (Polyline poly in currentLayer)
372             {
373                 Polyline[] offsets = poly.CreateOffsetPolyline(-offsetStep,
tolerance);
374
375                 foreach (Polyline offset in offsets)
376                 {
377                     if (!offset.Closed || offset.Points.Count < 3)
378                         continue;
379
380                     // Gera superfície entre curva atual e offset
381                     //Surface faixa =
SurfaceBuilderCopilot.BuildSurfaceBetweenCurves(poly.Points, offset.Points);
382                     var lower = Geometry3F.ToPointList(poly.Points);
383                     var upper = Geometry3F.ToPointList(offset.Points);
384                     Surface faixa =
SurfaceBuilderCopilot.BuildSurfaceBetweenCurves(lower, upper);
385
386                     // Reindexa pontos e faces
387                     for (int f = 0; f < faixa.Faces.Length; f++)
388                     {
389                         TriangleFace face = faixa.Faces[f];
390                         Point3F pa = faixa.Points[face.A];
391                         Point3F pb = faixa.Points[face.B];
392                         Point3F pc = faixa.Points[face.C];
393
394                         int ia = AddPoint(pa, allPoints, pointIndex);
395                         int ib = AddPoint(pb, allPoints, pointIndex);
396                         int ic = AddPoint(pc, allPoints, pointIndex);
397
398                         allFaces.Add(new TriangleFace(ia, ib, ic));
399                     }
400
401                     nextLayer.Add(offset);
402                 }
403             }
404
405             currentLayer = nextLayer;
406         }
407
408         // Cria superfície final consolidada
409         Surface finalSurface = new Surface
410         {
411             Points = allPoints,
412             Faces = allFaces.ToArray()
413         };
414
415         cadFile.Add(finalSurface);
416         _ui.ActiveView.ZoomToFit();
417         _ui.ActiveView.RefreshView();
418     }
419     private int AddPoint(Point3F p, Point3FArray points, Dictionary<Point3F, int>
indexMap)

```

```

420     {
421         int index;
422         if (!indexMap.TryGetValue(p, out index))
423         {
424             index = points.Count;
425             points.Add(p);
426             indexMap[p] = index;
427         }
428         return index;
429     }
430     private List<List<Point3F>> SimplifyAll(List<List<Point3F>> curves)
431     {
432         var result = new List<List<Point3F>>();
433         foreach (var curve in curves)
434             result.Add(PolylineSimplifier.SimplifyDouglasPeucker(curve, 0.001));
435         return result;
436     }
437
438     private string CreateUniqueLayer(string baseName)
439     {
440         int index = 1;
441         string layerName;
442         var cadFile = _ui.ActiveView.CADFile;
443
444         do
445         {
446             layerName = $"{baseName}{index:D4}";
447             index++;
448         }
449         while (cadFile.HasLayer(layerName));
450
451         cadFile.CreateLayer(layerName);
452         cadFile.SetActiveLayer(layerName);
453         return layerName;
454     }
455 }
456 using PluginSettings;
457
458 namespace MorphMuse.model
459 {
460     public class SettingsProfile
461     {
462         public string Name { get; set; } = "default";
463         public SettingsManager.Units Units { get; set; } =
SettingsManager.Units.Millimeters;
464         public float Tolerance { get; set; } = 0.01f;
465         public float OffsetStep { get; set; } = 0.5f;
466         public bool UseAdvancedMode { get; set; } = false;
467     }
468 }
469 using System;
470 using System.Collections.Generic;
471 using System.IO;
472 using Newtonsoft.Json;
473
474 public class PluginSettings
475 {
476     public string ActiveProfile { get; set; } = "default";
477     public Dictionary<string, SettingsProfile> Profiles { get; set; } = new();
478
479     private static string ConfigPath =>
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData),
"MorphMuse", "settings.json");
480
481     public void Save()
482     {
483         Directory.CreateDirectory(Path.GetDirectoryName(ConfigPath));
484         var json = JsonSerializer.Serialize(this, new JsonSerializerOptions {
WriteIndented = true });
485         File.WriteAllText(ConfigPath, json);
486     }
487
488     public static SettingsRepository Load()

```

```

489     {
490         if (!File.Exists(ConfigPath))
491             return new SettingsRepository();
492
493         var json = File.ReadAllText(ConfigPath);
494         return JsonSerializer.Deserialize<SettingsRepository>(json) ?? new
SettingsRepository();
495     }
496
497     public SettingsProfile GetActiveProfile()
498     {
499         return Profiles.ContainsKey(ActiveProfile) ? Profiles[ActiveProfile] : new
SettingsProfile();
500     }
501 }
502 using CamBam.Geom;
503 using CamBam.CAD;
504 using System.Collections.Generic;
505
506 namespace MorphMuse.Services
507 {
508     public static class SurfaceBuilderCopilot
509     {
510         public static Surface BuildSurfaceBetweenCurves(List<Point3F> lower,
List<Point3F> upper)
511         {
512             var points = new Point3FArray();
513             var pointIndex = new Dictionary<Point3F, int>();
514             var faces = new List<TriangleFace>();
515
516             if (Geometry3F.Distance(lower[0], lower[lower.Count - 1]) > 1e-6)
517                 lower.Add(lower[0]);
518
519             if (Geometry3F.Distance(upper[0], upper[upper.Count - 1]) > 1e-6)
520                 upper.Add(upper[0]);
521
522             int i = 0, j = 0;
523
524             while (i < lower.Count - 1 && j < upper.Count - 1)
525             {
526                 Point3F a = lower[i];
527                 Point3F b = upper[j];
528                 Point3F aNext = lower[i + 1];
529                 Point3F bNext = upper[j + 1];
530
531                 double dANextToB = Geometry3F.Distance(aNext, b);
532                 double dAtoBNext = Geometry3F.Distance(a, bNext);
533
534                 if (dANextToB < dAtoBNext)
535                 {
536                     AddTriangle(a, b, aNext, points, pointIndex, faces);
537                     i++;
538                 }
539                 else
540                 {
541                     AddTriangle(a, b, bNext, points, pointIndex, faces);
542                     j++;
543                 }
544             }
545
546             while (i < lower.Count - 1)
547             {
548                 AddTriangle(lower[i], upper[j], lower[i + 1], points, pointIndex,
faces);
549                 i++;
550             }
551
552             while (j < upper.Count - 1)
553             {
554                 AddTriangle(lower[i], upper[j], upper[j + 1], points, pointIndex,
faces);
555                 j++;
556             }

```



```

557
558         var surface = new Surface
559         {
560             Points = points,
561             Faces = faces.ToArray()
562         };
563
564         return surface;
565     }
566
567     public static void GenerateCapSurface(List<Point3F> topCurve, Point3F center,
Point3FArray points, Dictionary<Point3F, int> indexMap, List<TriangleFace> faces)
568     {
569         int centerIndex = AddPoint(center, points, indexMap);
570
571         for (int i = 0; i < topCurve.Count - 1; i++)
572         {
573             int ia = AddPoint(topCurve[i], points, indexMap);
574             int ib = AddPoint(topCurve[i + 1], points, indexMap);
575             faces.Add(new TriangleFace(centerIndex, ia, ib));
576         }
577
578         int iaLast = AddPoint(topCurve[topCurve.Count - 1], points, indexMap);
579         int ibFirst = AddPoint(topCurve[0], points, indexMap);
580         faces.Add(new TriangleFace(centerIndex, iaLast, ibFirst));
581     }
582     public static void GenerateLateralSurface(List<List<Point3F>>
simplifiedCurves, Point3FArray points, Dictionary<Point3F, int> indexMap,
List<TriangleFace> faces)
583     {
584         for (int i = 0; i < simplifiedCurves.Count - 1; i++)
585         {
586             var lower = simplifiedCurves[i];
587             var upper = simplifiedCurves[i + 1];
588
589             Surface partialSurface =
SurfaceBuilderCopilot.BuildSurfaceBetweenCurves(lower, upper);
590
591             foreach (var face in partialSurface.Faces)
592             {
593                 Point3F pa = partialSurface.Points[face.A];
594                 Point3F pb = partialSurface.Points[face.B];
595                 Point3F pc = partialSurface.Points[face.C];
596
597                 int ia = AddPoint(pa, points, indexMap);
598                 int ib = AddPoint(pb, points, indexMap);
599                 int ic = AddPoint(pc, points, indexMap);
600
601                 faces.Add(new TriangleFace(ia, ib, ic));
602             }
603         }
604     }
605
606     private static void AddTriangle(Point3F a, Point3F b, Point3F c,
Point3FArray points,
607                                     Dictionary<Point3F, int> pointIndex,
List<TriangleFace> faces)
608     {
609         if (IsDegenerate(a, b, c)) return;
610
611         int ia = AddPoint(a, points, pointIndex);
612         int ib = AddPoint(b, points, pointIndex);
613         int ic = AddPoint(c, points, pointIndex);
614
615         // Orientação fixa: sempre a → b → c
616         faces.Add(new TriangleFace(ia, ic, ib));
617     }
618
619     private static int AddPoint(Point3F p, Point3FArray points,
Dictionary<Point3F, int> indexMap)
620     {
621         if (!indexMap.TryGetValue(p, out int index))
622         {
623             index = points.Length;
624             points.Add(p);
625             indexMap.Add(p, index);
626         }
627         return index;
628     }

```

```

625         {
626             index = points.Count;
627             points.Add(p);
628             indexMap[p] = index;
629         }
630         return index;
631     }
632
633     private static bool IsDegenerate(Point3F a, Point3F b, Point3F c)
634     {
635         var ab = Geometry3F.FromPoints(a, b);
636         var ac = Geometry3F.FromPoints(a, c);
637         var cross = Geometry3F.Cross(ab, ac);
638         return Geometry3F.Length(cross) * 0.5 < 1e-6;
639     }
640 }
641
642 using CamBam.CAD;
643 using CamBam.Geom;
644 using System;
645 using System.Collections.Generic;
646 using System.Linq;
647
648 namespace MorphMuse.Services
649 {
650     public static class SurfaceBuilderCopilot
651     {
652         public static Surface BuildSurfaceBetweenCurves(List<Point3F> lower,
653 List<Point3F> upper)
654         {
655             var points = new Point3FArray();
656             var pointIndex = new Dictionary<Point3F, int>();
657             var faces = new List<TriangleFace>();
658             //CamBam.ThisApplication.AddLogMessage($"lower.Count= {lower.Count:F4}");
659             CamBam.ThisApplication.AddLogMessage($"[Copilot] --- Início da
triangulação ---");
660             CamBam.ThisApplication.AddLogMessage($"[Copilot] lower.Count =
{lower.Count}");
661             CamBam.ThisApplication.AddLogMessage($"[Copilot] upper.Count =
{upper.Count}");
662
663             int maxLog = Math.Min(5, Math.Min(lower.Count, upper.Count));
664             for (int k = 0; k < maxLog; k++)
665             {
666                 var pl = lower[k];
667                 var pu = upper[k];
668                 CamBam.ThisApplication.AddLogMessage(
669                     $"[Copilot] Ponto {k}: lower=({pl.X:F3}, {pl.Y:F3}, {pl.Z:F3}) |
upper=({pu.X:F3}, {pu.Y:F3}, {pu.Z:F3})");
670             }
671             if (Geometry3F.Distance(lower[0], lower[lower.Count - 1]) > 1e-6)
672                 lower.Add(lower[0]);
673
674             if (Geometry3F.Distance(upper[0], upper[upper.Count - 1]) > 1e-6)
675                 upper.Add(upper[0]);
676
677             int i = 0, j = 0;
678
679             while (i < lower.Count - 1 && j < upper.Count - 1)
680             {
681                 //Point3F a = lower[i];
682                 //Point3F b = upper[j];
683                 //Point3F aNext = lower[i + 1];
684                 //Point3F bNext = upper[j + 1];
685                 //double dANextToB = Geometry3F.Distance(aNext, b);
686                 //double dAtoBNext = Geometry3F.Distance(a, bNext);
687
688                 //if (dANextToB < dAtoBNext)
689                 //{
690                     AddTriangle(a, b, aNext, points, pointIndex, faces);
691                     i++;
692                 //}
693             }
694             //else

```

```

693         //{
694         //     AddTriangle(a, b, bNext, points, pointIndex, faces);
695         //     j++;
696         //}
697         double fracLower = i / (double) (lower.Count - 1);
698         double fracUpper = j / (double) (upper.Count - 1);
699         //CamBam.ThisApplication.AddLogMessage($"lower.Count=
{lower.Count:F4}");
700
701         if (fracLower <= fracUpper && i < lower.Count - 1)
702         {
703             AddTriangle(lower[i], upper[j], lower[i + 1], points, pointIndex,
faces);
704             i++;
705         }
706         else if (j < upper.Count - 1)
707         {
708             AddTriangle(lower[i], upper[j], upper[j + 1], points, pointIndex,
faces);
709             j++;
710         }
711     }
712
713     while (i < lower.Count - 1 || j < upper.Count - 1)
714     {
715         if (i < lower.Count - 1 && (j == upper.Count - 1 || i /
(double) (lower.Count - 1) <= j / (double) (upper.Count - 1)))
716         {
717             AddTriangle(lower[i], upper[j], lower[i + 1], points, pointIndex,
faces);
718             i++;
719         }
720         else if (j < upper.Count - 1)
721         {
722             AddTriangle(lower[i], upper[j], upper[j + 1], points, pointIndex,
faces);
723             j++;
724         }
725     }
726
727     var surface = new Surface
728     {
729         Points = points,
730         Faces = faces.ToArray()
731     };
732
733     return surface;
734 }
735
736 public static void GenerateCapSurface(List<Point3F> topCurve, Point3F center,
Point3FArray points, Dictionary<Point3F, int> indexMap, List<TriangleFace> faces)
737 {
738     int centerIndex = AddPoint(center, points, indexMap);
739
740     for (int i = 0; i < topCurve.Count - 1; i++)
741     {
742         int ia = AddPoint(topCurve[i], points, indexMap);
743         int ib = AddPoint(topCurve[i + 1], points, indexMap);
744         faces.Add(new TriangleFace(centerIndex, ia, ib));
745     }
746
747     int iaLast = AddPoint(topCurve[topCurve.Count - 1], points, indexMap);
748     int ibFirst = AddPoint(topCurve[0], points, indexMap);
749     faces.Add(new TriangleFace(centerIndex, iaLast, ibFirst));
750 }
751 public static void GenerateLateralSurface(List<List<Point3F>>
simplifiedCurves, Point3FArray points, Dictionary<Point3F, int> indexMap,
List<TriangleFace> faces)
752 {
753     for (int i = 0; i < simplifiedCurves.Count - 1; i++)
754     {
755         var lower = simplifiedCurves[i];
756         var upper = simplifiedCurves[i + 1];

```

```

757
758         AlignCurveToPrevious(lower, upper); // Align upper and lower points
759
760         Surface partialSurface =
SurfaceBuilderCopilot.BuildSurfaceBetweenCurves(lower, upper);
761
762         foreach (var face in partialSurface.Faces)
763         {
764             Point3F pa = partialSurface.Points[face.A];
765             Point3F pb = partialSurface.Points[face.B];
766             Point3F pc = partialSurface.Points[face.C];
767
768             int ia = AddPoint(pa, points, indexMap);
769             int ib = AddPoint(pb, points, indexMap);
770             int ic = AddPoint(pc, points, indexMap);
771
772             faces.Add(new TriangleFace(ia, ib, ic));
773         }
774     }
775 }
776
777
778 private static void AddTriangle(Point3F a, Point3F b, Point3F c,
779                                Point3FArray points,
780                                Dictionary<Point3F, int> pointIndex,
781                                List<TriangleFace> faces)
782 {
783     if (IsDegenerate(a, b, c)) return;
784
785     int ia = AddPoint(a, points, pointIndex);
786     int ib = AddPoint(b, points, pointIndex);
787     int ic = AddPoint(c, points, pointIndex);
788
789     // Orientação fixa: sempre a → c → b
790     faces.Add(new TriangleFace(ia, ic, ib));
791 }
792
793 private static int AddPoint(Point3F p, Point3FArray points,
Dictionary<Point3F, int> indexMap)
794 {
795     if (!indexMap.TryGetValue(p, out int index))
796     {
797         index = points.Count;
798         points.Add(p);
799         indexMap[p] = index;
800     }
801     return index;
802 }
803
804 private static bool IsDegenerate(Point3F a, Point3F b, Point3F c)
805 {
806     var ab = Geometry3F.FromPoints(a, b);
807     var ac = Geometry3F.FromPoints(a, c);
808     var cross = Geometry3F.Cross(ab, ac);
809     return Geometry3F.Length(cross) * 0.5 < 1e-6;
810 }
811 private static bool ChooseAdvanceByArea(Point3F a, Point3F b, Point3F aNext,
Point3F bNext)
812 {
813     double areaA = TriangleArea(a, b, aNext);
814     double areaB = TriangleArea(a, b, bNext);
815     return areaA >= areaB; // true → avança i (lower), false → avança j
(upper)
816 }
817
818 private static double TriangleArea(Point3F a, Point3F b, Point3F c)
819 {
820     var ab = Geometry3F.FromPoints(a, b);
821     var ac = Geometry3F.FromPoints(a, c);
822     var cross = Geometry3F.Cross(ab, ac);
823     return Geometry3F.Length(cross) * 0.5;
824 }
825 private static bool ChooseAdvanceByAngle(Point3F a, Point3F b, Point3F aNext,

```

```

Point3F bNext)
{
    var ab = Geometry3F.FromPoints(a, b);
    var va = Geometry3F.FromPoints(a, aNext);
    var vb = Geometry3F.FromPoints(b, bNext);

    double angleA = AngleBetween(ab, va);
    double angleB = AngleBetween(ab, vb);

    return angleA <= angleB; // menor ângulo → avança i (lower)
}

private static double AngleBetween(Vector3F u, Vector3F v)
{
    double dot = u.X * v.X + u.Y * v.Y + u.Z * v.Z;
    double lu = Geometry3F.Length(u);
    double lv = Geometry3F.Length(v);
    if (lu < 1e-8 || lv < 1e-8) return Math.PI;
    double cos = Math.Max(-1.0, Math.Min(1.0, dot / (lu * lv)));
    return Math.Acos(cos);
}

public static void AlignCurveToPrevious(List<Point3F> previous, List<Point3F>
current)
{
    int n = current.Count;
    int m = previous.Count;
    if (n < 3 || m < 3) return;

    int minCount = Math.Min(n, m);
    double bestScore = double.MaxValue;
    int bestOffset = 0;

    for (int offset = 0; offset < n; offset++)
    {
        double score = 0;
        for (int i = 0; i < minCount; i++)
        {
            int j = (i + offset) % n;
            score += Geometry3F.Distance(previous[i], current[j]);
        }

        if (score < bestScore)
        {
            bestScore = score;
            bestOffset = offset;
        }
    }

    if (bestOffset > 0)
    {
        var rotated = new List<Point3F>();
        rotated.AddRange(current.Skip(bestOffset));
        rotated.AddRange(current.Take(bestOffset));
        current.Clear();
        current.AddRange(rotated);

        CamBam.ThisApplication.AddLogMessage($"[Copilot] Curva rotacionada
por {bestOffset} posições.");
    }
    else
    {
        CamBam.ThisApplication.AddLogMessage($"[Copilot] Curva já estava
alinhada (offset = 0).");
    }
}

}

using CamBam.Geom;
using MorphMuse.Services;

public class Triangle
{
    public Point3F A;

```

```

895     public Point3F B;
896     public Point3F C;
897
898     public Triangle(Point3F a, Point3F b, Point3F c)
899     {
900         this.A = a;
901         this.B = b;
902         this.C = c;
903     }
904
905     public Point3F GetCenter()
906     {
907         return new Point3F(
908             (A.X + B.X + C.X) / 3f,
909             (A.Y + B.Y + C.Y) / 3f,
910             (A.Z + B.Z + C.Z) / 3f
911         );
912     }
913
914     public Vector3F GetNormal()
915     {
916         Vector3F u = new Vector3F(B.X - A.X, B.Y - A.Y, B.Z - A.Z);
917         Vector3F v = new Vector3F(C.X - A.X, C.Y - A.Y, C.Z - A.Z);
918
919         Vector3F normal = Geometry3F.Cross(u, v);
920         return Geometry3F.Normalize(normal);
921     }
922 }
923 using CamBam.Geom;
924 using System.Collections.Generic;
925
926 namespace MorphMuse.Model
927 {
928     internal class Triangle
929     {
930         public Point3F A { get; set; }
931         public Point3F B { get; set; }
932         public Point3F C { get; set; }
933
934         public Triangle(Point3F a, Point3F b, Point3F c)
935         {
936             this.A = a;
937             this.B = b;
938             this.C = c;
939         }
940     }
941
942     internal class MeshGenerator
943     {
944         public List<Triangle> GenerateMesh(List<List<Point3F>> layers)
945         {
946             var triangles = new List<Triangle>();
947
948             for (int i = 0; i < layers.Count - 1; i++)
949             {
950                 var lower = layers[i];
951                 var upper = layers[i + 1];
952
953                 int count = System.Math.Min(lower.Count, upper.Count);
954
955                 for (int j = 0; j < count - 1; j++)
956                 {
957                     // Triângulo 1
958                     triangles.Add(new Triangle(
959                         lower[j],
960                         upper[j],
961                         upper[j + 1]
962                     ));
963
964                     // Triângulo 2
965                     triangles.Add(new Triangle(
966                         lower[j],
967                         upper[j + 1],

```

```

968         lower[j + 1]
969     ));
970 }
971
972 // Fecha o loop se for polilinha fechada
973 if (lower.Count > 2 && upper.Count > 2)
974 {
975     triangles.Add(new Triangle(
976         lower[count - 1],
977         upper[count - 1],
978         upper[0]
979     ));
980
981     triangles.Add(new Triangle(
982         lower[count - 1],
983         upper[0],
984         lower[0]
985     ));
986 }
987 }
988
989 return triangles;
990 }
991 }
992 }
993 // <autogenerated />
994 using System;
995 using System.Reflection;
996 [assembly:
997     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.8",
998         FrameworkDisplayName = ".NET Framework 4.8")]
999 // <autogenerated />
1000 using System;
1001 using System.Reflection;
1002 [assembly:
1003     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.6",
1004         FrameworkDisplayName = ".NET Framework 4.6")]
1005 // <autogenerated />
1006 using System;
1007 using System.Reflection;
1008 [assembly:
1009     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.8",
1010         FrameworkDisplayName = ".NET Framework 4.8")]
1011 // <autogenerated />
1012 using System;
1013 using System.Reflection;
1014 [assembly:
1015     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.6",
1016         FrameworkDisplayName = ".NET Framework 4.6")]
1017 // <autogenerated />
1018 using System;
1019 using System.Reflection;
1020 [assembly:
1021     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.8",
1022         FrameworkDisplayName = ".NET Framework 4.8")]
1023 // <autogenerated />
1024 using System;
1025 using System.Reflection;
1026 [assembly:
1027     global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.6",
1028         FrameworkDisplayName = ".NET Framework 4.6")]

```

```

1025 // <autogenerated />
1026 using System;
1027 using System.Reflection;
1028 [assembly:
global::System.Runtime.Versioning.TargetFrameworkAttribute(".NETFramework,Version=v4.8
", FrameworkDisplayName = ".NET Framework 4.8")]
1029 using System.Reflection;
1030 using System.Runtime.CompilerServices;
1031 using System.Runtime.InteropServices;
1032
1033 // General Information about an assembly is controlled through the following
1034 // set of attributes. Change these attribute values to modify the information
1035 // associated with an assembly.
1036 [assembly: AssemblyTitle("MorphMuse")]
1037 [assembly: AssemblyDescription("Cambam Plugin")]
1038 [assembly: AssemblyConfiguration("")]
1039 [assembly: AssemblyCompany("Ricardo Y Maeda")]
1040 [assembly: AssemblyProduct("MorphMuse")]
1041 [assembly: AssemblyCopyright("Copyright © 2025")]
1042 [assembly: AssemblyTrademark("")]
1043 [assembly: AssemblyCulture("")]
1044
1045 // Setting ComVisible to false makes the types in this assembly not visible
1046 // to COM components. If you need to access a type in this assembly from
1047 // COM, set the ComVisible attribute to true on that type.
1048 [assembly: ComVisible(false)]
1049
1050 // The following GUID is for the ID of the typelib if this project is exposed to COM
1051 [assembly: Guid("fc161397-6c90-443d-a75a-b298915b7946")]
1052
1053 // Version information for an assembly consists of the following four values:
1054 //
1055 //      Major Version
1056 //      Minor Version
1057 //      Build Number
1058 //      Revision
1059 //
1060 [assembly: AssemblyVersion("1.3.115.0")]
1061 [assembly: AssemblyFileVersion("1.3.115.0")]
1062 [assembly: AssemblyInformationalVersion("1.3.115.0")]
1063 //-----
1064 // <auto-generated>
1065 //      This code was generated by a tool.
1066 //      Runtime Version:4.0.30319.42000
1067 //
1068 //      Changes to this file may cause incorrect behavior and will be lost if
1069 //      the code is regenerated.
1070 // </auto-generated>
1071 //-----
1072
1073 namespace MorphMuse.Properties {
1074     using System;
1075
1076     /// <summary>
1077     ///      A strongly-typed resource class, for looking up localized strings, etc.
1078     /// </summary>
1079     /// This class was auto-generated by the StronglyTypedResourceBuilder
1080     /// class via a tool like ResGen or Visual Studio.
1081     /// To add or remove a member, edit your .ResX file then rerun ResGen
1082     /// with the /str option, or rebuild your VS project.
1083
1084     [global::System.CodeDom.Compiler.GeneratedCodeAttribute("System.Resources.Tools.Strong
lyTypedResourceBuilder", "17.0.0.0")]
1085     [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
1086     [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
1087     internal class Resources {
1088
1089         private static global::System.Resources.ResourceManager resourceMan;
1090
1091         private static global::System.Globalization.CultureInfo resourceCulture;
1092
1093

```



```

[global::System.Diagnostics.CodeAnalysis.SuppressMessageAttribute("Microsoft.Performance", "CA1811:AvoidUncalledPrivateCode")]
1094     internal Resources() {
1095     }
1096
1097     /// <summary>
1098     ///     Returns the cached ResourceManager instance used by this class.
1099     /// </summary>
1100
1101     [global::System.ComponentModel.EditorBrowsableAttribute(global::System.ComponentModel.
1102     EditorBrowsableState.Advanced)]
1103     internal static global::System.Resources.ResourceManager ResourceManager {
1104     get {
1105         if (object.ReferenceEquals(resourceMan, null)) {
1106             global::System.Resources.ResourceManager temp = new
1107             global::System.Resources.ResourceManager("MorphMuse.Properties.Resources",
1108             typeof(Resources).Assembly);
1109             resourceMan = temp;
1110         }
1111         return resourceMan;
1112     }
1113
1114     /// <summary>
1115     ///     Overrides the current thread's CurrentUICulture property for all
1116     ///     resource lookups using this strongly typed resource class.
1117     /// </summary>
1118
1119     [global::System.ComponentModel.EditorBrowsableAttribute(global::System.ComponentModel.
1120     EditorBrowsableState.Advanced)]
1121     internal static global::System.Globalization.CultureInfo Culture {
1122     get {
1123         return resourceCulture;
1124     }
1125     set {
1126         resourceCulture = value;
1127     }
1128 }
1129
1130 //-----
1131 // <auto-generated>
1132 //     This code was generated by a tool.
1133 //     Runtime Version:4.0.30319.42000
1134 //
1135 //     Changes to this file may cause incorrect behavior and will be lost if
1136 //     the code is regenerated.
1137 // </auto-generated>
1138 //-----
1139
1140 namespace MorphMuse.Properties {
1141
1142     [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
1143
1144     [global::System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.VisualStudio.Editor
1145     s.SettingsDesigner.SettingsSingleFileGenerator", "17.14.0.0")]
1146     internal sealed partial class Settings :
1147     global::System.Configuration.ApplicationSettingsBase {
1148
1149     private static Settings defaultInstance =
1150     ((Settings)(global::System.Configuration.ApplicationSettingsBase.Synchronized(new
1151     Settings())));
1152
1153     public static Settings Default {
1154     get {
1155         return defaultInstance;
1156     }
1157 }
1158 }
1159
1160 using CamBam.CAD;
1161 using CamBam.Geom;

```

```

1154 using System;
1155 using System.Collections.Generic;
1156
1157 namespace MorphMuse.Services
1158 {
1159     internal class ConvexCapBuilder
1160     {
1161
1162         public static List<Surface> CloseNonConvexPolyline(List<Point3F> polyline,
1163 double simplificationTolerance)
1164         {
1165             var simplified = PolylineSimplifier.SimplifyDouglasPeucker(polyline,
1166 simplificationTolerance);
1167             var convexGroups = SegmentIntoConvexSubPolylines(simplified);
1168             var surfaces = new List<Surface>();
1169
1170             foreach (var group in convexGroups)
1171             {
1172                 Point3F center = Geometry3F.GetCentroid(group);
1173                 var points = new Point3FArray();
1174                 var indexMap = new Dictionary<Point3F, int>();
1175                 var faces = new List<TriangleFace>();
1176
1177                 int centerIndex = Geometry3F.AddPoint(center, points, indexMap);
1178                 for (int i = 0; i < group.Count; i++)
1179                 {
1180                     int ia = Geometry3F.AddPoint(group[i], points, indexMap);
1181                     int ib = Geometry3F.AddPoint(group[(i + 1) % group.Count],
1182 points, indexMap);
1183                     faces.Add(new TriangleFace(centerIndex, ia, ib));
1184                 }
1185                 surfaces.Add(new Surface { Points = points, Faces = faces.ToArray()
1186 });
1187             }
1188             return surfaces;
1189         }
1190
1191         public static List<List<Point3F>> SegmentIntoConvexSubPolylines(List<Point3F>
1192 curve)
1193         {
1194             var groups = new List<List<Point3F>>();
1195             var currentGroup = new List<Point3F>();
1196
1197             int count = curve.Count;
1198             for (int i = 0; i < count; i++)
1199             {
1200                 Point3F prev = curve[(i - 1 + count) % count];
1201                 Point3F curr = curve[i];
1202                 Point3F next = curve[(i + 1) % count];
1203
1204                 Vector3F v1 = Geometry3F.FromPoints(prev, curr);
1205                 Vector3F v2 = Geometry3F.FromPoints(curr, next);
1206                 double angle = Geometry3F.AngleBetween(v1, v2) * (180.0 / Math.PI);
1207
1208                 currentGroup.Add(curr);
1209
1210                 if (angle > 180.0)
1211                 {
1212                     if (currentGroup.Count >= 3)
1213                         groups.Add(new List<Point3F>(currentGroup));
1214                     currentGroup.Clear();
1215                 }
1216             }
1217
1218             if (currentGroup.Count >= 3)
1219                 groups.Add(currentGroup);
1220
1221             return groups;
1222         }
1223     }
1224 }

```

```

1222 using CamBam.CAD;
1223 using CamBam.Geom;
1224 using MorphMuse.Services;
1225 using System;
1226 using System.Collections.Generic;
1227
1228 public static class CurveSampler
1229 {
1230     public static PointList CreatePointlistFromPolylineStepCopilot(Polyline curve,
1231 double StepMax)
1232     {
1233         var points = new PointList();
1234
1235         // Clone and apply transformation, if any
1236         var polyline = (Polyline)curve.Clone();
1237         if (polyline.ApplyTransformation())
1238         {
1239             polyline.Transform = Matrix4x4F.Identity;
1240         }
1241
1242         // Extract the segments and arcs from the polyline
1243         Entity[] primitives = polyline.ToPrimitives();
1244
1245         foreach (var primitive in primitives)
1246         {
1247             if (primitive is Arc arc)
1248             {
1249                 // Calculate the number of points based on the radius and StepMax
1250                 double arcLength = Math.Abs(arc.Sweep) * Math.PI / 180.0 * arc.Radius;
1251                 int steps = Math.Max(2, (int)Math.Ceiling(arcLength / StepMax));
1252
1253                 double angleStep = arc.Sweep / steps;
1254                 for (int i = 0; i <= steps; i++)
1255                 {
1256                     double angleDeg = arc.Start + i * angleStep;
1257                     double angleRad = angleDeg * Math.PI / 180.0;
1258
1259                     double x = arc.Point.X + arc.Radius * Math.Cos(angleRad);
1260                     double y = arc.Point.Y + arc.Radius * Math.Sin(angleRad);
1261                     points.Add(new Point3F(x, y, arc.Point.Z));
1262                 }
1263             }
1264             else if (primitive is Line line)
1265             {
1266                 // Add only the endpoints of the line
1267                 if (line.Points.Count > 0)
1268                 {
1269                     points.Add(line.Points[0]);
1270                     points.Add(line.Points[line.Points.Count - 1]);
1271                 }
1272             }
1273
1274             // Apply final transformation, if necessary
1275             if (!polyline.Transform.IsIdentity())
1276             {
1277                 points.ApplyTransformation(polyline.Transform);
1278             }
1279
1280             // Remove duplicate if curve is closed
1281             if (points.Points.Count > 1 && Point3F.Match(points.Points[0],
1282 points.Points[points.Points.Count - 1]))
1283             {
1284                 points.Points.RemoveAt(points.Points.Count - 1);
1285             }
1286
1287             return points;
1288         }
1289
1290         /// <summary>
1291         /// Generates sampled points from a set of ordered contours, using a generatrix
1292 and density parameters.
1293         /// Each contour is sampled according to its spacing and the provided density.

```

```

1292     /// </summary>
1293     /// <param name="orderedContours">Contours ordered by generatrix.</param>
1294     /// <param name="simplifiedGeratriz">Simplified generatrix points.</param>
1295     /// <param name="baseDensity">Base density for sampling.</param>
1296     /// <param name="minStep">Minimum allowed step size.</param>
1297     /// <returns>A list of lists of sampled points for each contour.</returns>
1298     public static List<List<Point3F>> GenerateSampledPointsFromContours(
1299         List<List<Polyline>> orderedContours,
1300         List<Point3F> simplifiedGeratriz,
1301         double baseDensity,
1302         double minStep)
1303     {
1304         var sampledCurves = new List<List<Point3F>>();
1305
1306         // Iterate through each contour group.
1307         for (int i = 0; i < orderedContours.Count; i++)
1308         {
1309             List<Polyline> curves = orderedContours[i];
1310             double step = baseDensity;
1311
1312             // Adjust the sampling step based on the spacing between generatrix
1313             points. if (i < simplifiedGeratriz.Count - 1)
1314             {
1315                 Point3F p1 = simplifiedGeratriz[i];
1316                 Point3F p2 = simplifiedGeratriz[i + 1];
1317                 Vector3F delta = new Vector3F(p2.X - p1.X, p2.Y - p1.Y, p2.Z - p1.Z);
1318
1319                 double spacing = Geometry3F.Length(delta);
1320                 step = spacing * baseDensity;
1321                 // Ensure the step is not below the minimum allowed.
1322                 if (step < minStep)
1323                 {
1324                     step = minStep;
1325                 }
1326             }
1327
1328             // Sample each polyline in the current contour group.
1329             for (int j = 0; j < curves.Count; j++)
1330             {
1331                 Polyline curve = curves[j];
1332                 PointList rawPoints = CreatePointlistFromPolylineStepCopilot(curve,
1333                 step);
1334
1335                 List<Point3F> converted = new List<Point3F>();
1336
1337                 // Convert the PointList to a standard List<Point3F>.
1338                 for (int k = 0; k < rawPoints.Points.Count; k++)
1339                 {
1340                     converted.Add(rawPoints.Points[k]);
1341                 }
1342                 sampledCurves.Add(converted);
1343             }
1344         }
1345
1346         return sampledCurves;
1347     }
1348 }
1349 using CamBam.Geom;
1350 using System;
1351 using System.Collections.Generic;
1352
1353 namespace MorphMuse.Services
1354 {
1355     public static class EarClippingTriangulator
1356     {
1357         /// <summary>
1358         /// Triangula um pol -gono simples (pode ser c ncavo) usando o algoritmo de
1359         Ear Clipping.
1360         /// Assume que todos os pontos est o no mesmo plano.
1361         /// </summary>
1362         public static List<TriangleFace> Triangulate(List<Point3F> polygon,

```

```

Point3FArray points, Dictionary<Point3F, int> indexMap)
{
    var faces = new List<TriangleFace>();
    if (polygon.Count < 3) return faces;

    // Criar uma lista de Índices para trabalhar
    List<int> indices = new List<int>();
    for (int i = 0; i < polygon.Count; i++)
    {
        indices.Add(i);
    }

    // Determinar a orientação do polígono (sentido horário ou
anti-horário)
    // Usamos a soma das áreas assinadas (Shoelace formula)
    bool isClockwise = IsClockwise(polygon);

    int iterations = 0;
    while (indices.Count > 3 && iterations < 2000) // Guardrail aumentado
    {
        bool earFound = false;
        for (int i = 0; i < indices.Count; i++)
        {
            int prevIdx = indices[(i - 1 + indices.Count) % indices.Count];
            int currIdx = indices[i];
            int nextIdx = indices[(i + 1) % indices.Count];

            if (IsEar(prevIdx, currIdx, nextIdx, indices, polygon,
isClockwise))
            {
                // Adicionar o triângulo
                int ia = Geometry3F.AddPoint(polygon[prevIdx], points,
indexMap);
                int ib = Geometry3F.AddPoint(polygon[currIdx], points,
indexMap);
                int ic = Geometry3F.AddPoint(polygon[nextIdx], points,
indexMap);

                // Orientação ajustada para normais corretas (ia, ib, ic)
                // O usuário mencionou que precisou trocar b e c, então
usamos a ordem que gera a normal correta.
                faces.Add(new TriangleFace(ia, ib, ic));

                // Remover o vértice "orelha"
                indices.RemoveAt(i);
                earFound = true;
                break;
            }
        }

        if (!earFound) break;
        iterations++;
    }

    // Adicionar o último triângulo restante
    if (indices.Count == 3)
    {
        int ia = Geometry3F.AddPoint(polygon[indices[0]], points, indexMap);
        int ib = Geometry3F.AddPoint(polygon[indices[1]], points, indexMap);
        int ic = Geometry3F.AddPoint(polygon[indices[2]], points, indexMap);
        faces.Add(new TriangleFace(ia, ib, ic));
    }

    return faces;
}

private static bool IsEar(int pIdx, int cIdx, int nIdx, List<int> indices,
List<Point3F> polygon, bool isClockwise)
{
    Point3F a = polygon[pIdx];
    Point3F b = polygon[cIdx];
    Point3F c = polygon[nIdx];

```

```

1427 // 1. Verificar se o Ângulo Â convexo em rela  o ao interior do
1428 pol gono
1429 if (!IsConvex(a, b, c, isClockwise)) return false;
1430
1431 // 2. Verificar se algum outro ponto do pol gono est  dentro do
1432 tri ngulo (a, b, c)
1433 // Usamos uma margem de erro pequena (epsilon) para evitar problemas de
1434 precis o num rica
1435 foreach (int idx in indices)
1436 {
1437     if (idx == pIdx || idx == cIdx || idx == nIdx) continue;
1438     if (IsPointInTriangle(polygon[idx], a, b, c)) return false;
1439 }
1440
1441 return true;
1442 }
1443
1444 private static bool IsConvex(Point3F a, Point3F b, Point3F c, bool
isClockwise)
1445 {
1446     // Produto vetorial 2D (z-component)
1447     double crossProduct = (b.X - a.X) * (c.Y - b.Y) - (b.Y - a.Y) * (c.X -
b.X);
1448
1449     // Se for anti-hor rio (isClockwise = false), crossProduct > 0   convexo
1450     // Se for hor rio (isClockwise = true), crossProduct < 0   convexo
1451     if (isClockwise) return crossProduct < -1e-9;
1452     return crossProduct > 1e-9;
1453 }
1454
1455 private static bool IsClockwise(List<Point3F> polygon)
1456 {
1457     double area = 0;
1458     for (int i = 0; i < polygon.Count; i++)
1459     {
1460         Point3F p1 = polygon[i];
1461         Point3F p2 = polygon[(i + 1) % polygon.Count];
1462         area += (p2.X - p1.X) * (p2.Y + p1.Y);
1463     }
1464     // Em coordenadas de tela (Y para baixo), area > 0   hor rio.
1465     // Em coordenadas cartesianas padr o (CamBam), area > 0   hor rio.
1466     return area > 0;
1467 }
1468
1469 private static bool IsPointInTriangle(Point3F p, Point3F a, Point3F b,
Point3F c)
1470 {
1471     // Coordenadas baric nticas com toler ncia para precis o num rica
1472     double det = (b.Y - c.Y) * (a.X - c.X) + (c.X - b.X) * (a.Y - c.Y);
1473     if (Math.Abs(det) < 1e-12) return false; // Tri ngulo degenerado
1474
1475     double baryA = ((b.Y - c.Y) * (p.X - c.X) + (c.X - b.X) * (p.Y - c.Y)) /
det;
1476     double baryB = ((c.Y - a.Y) * (p.X - c.X) + (a.X - c.X) * (p.Y - c.Y)) /
det;
1477     double baryC = 1 - baryA - baryB;
1478
1479     double epsilon = -1e-9;
1480     return baryA > epsilon && baryB > epsilon && baryC > epsilon;
1481 }
1482 }
1483
1484 using CamBam.CAD;
1485 using CamBam.Geom;
1486 using System.Collections.Generic;
1487
1488 namespace MorphMuse.Services
1489 {
1490     public class FanCapGenerator
1491     {
1492         static public void FindConvergenceCentersAndCurves(
            Polyline closedBase,
            double minOffset,

```

```

1492         double maxOffset,
1493         float tolerance,
1494         out List<Point3F> centers,
1495         out List<Polyline> finalCurves)
1496     {
1497         centers = new List<Point3F>();
1498         finalCurves = new List<Polyline>();
1499
1500         double low = minOffset;
1501         double high = maxOffset;
1502
1503         while ((high - low) > 0.01)
1504         {
1505             double mid = (low + high) / 2.0;
1506             Polyline[] offsets = closedBase.CreateOffsetPolyline(-mid, tolerance);
1507
1508             if (offsets == null || offsets.Length == 0)
1509             {
1510                 high = mid;
1511             }
1512             else
1513             {
1514                 low = mid;
1515                 centers.Clear();
1516                 finalCurves.Clear();
1517
1518                 for (int i = 0; i < offsets.Length; i++)
1519                 {
1520                     Polyline poly = offsets[i];
1521                     if (poly.Closed && poly.Points.Count >= 3)
1522                     {
1523                         finalCurves.Add(poly);
1524                         centers.Add(poly.GetCentroid()); // Usa API do CamBam
1525                     }
1526                 }
1527             }
1528         }
1529     }
1530     static public List<TriangleFace> GenerateFanCapForMultiple(
1531         List<Polyline> curves,
1532         List<Point3F> centers,
1533         Point3FArray points,
1534         Dictionary<Point3F, int> pointIndex)
1535     {
1536         List<TriangleFace> faces = new List<TriangleFace>();
1537         int count = System.Math.Min(curves.Count, centers.Count);
1538
1539         for (int i = 0; i < count; i++)
1540         {
1541             Polyline curve = curves[i];
1542             Point3F center = centers[i];
1543
1544             if (!curve.Closed || curve.Points.Count < 3)
1545                 continue;
1546
1547             int ic = Geometry3F.AddPoint(center, points, pointIndex);
1548
1549             for (int j = 0; j < curve.Points.Count - 1; j++)
1550             {
1551                 int ia = Geometry3F.AddPoint(curve.Points[j].Point, points,
pointIndex);
1552                 int ib = Geometry3F.AddPoint(curve.Points[j + 1].Point, points,
pointIndex);
1553                 faces.Add(new TriangleFace(ic, ia, ib));
1554             }
1555
1556             int iaLast = Geometry3F.AddPoint(curve.Points[curve.Points.Count -
1].Point, points, pointIndex);
1557             int ibFirst = Geometry3F.AddPoint(curve.Points[0].Point, points,
pointIndex);
1558             faces.Add(new TriangleFace(ic, iaLast, ibFirst));
1559         }
1560     }

```

```

1561         return faces;
1562     }
1563
1564     public static Point3F CalculateCentroid(Polyline poly)
1565     {
1566         float cx = 0f;
1567         float cy = 0f;
1568         float cz = 0f;
1569
1570         for (int i = 0; i < poly.Points.Count; i++)
1571         {
1572             Point3F pt = poly.Points[i].Point;
1573             cx += (float)pt.X;
1574             cy += (float)pt.Y;
1575             cz += (float)pt.Z;
1576         }
1577
1578         int count = poly.Points.Count;
1579         return new Point3F(cx / count, cy / count, cz / count);
1580     }
1581
1582     public static int AddPoint(Point3F p, Point3FArray points,
Dictionary<Point3F, int> indexMap)
1583     {
1584         int index;
1585         if (!indexMap.TryGetValue(p, out index))
1586         {
1587             index = points.Count;
1588             points.Add(p);
1589             indexMap[p] = index;
1590         }
1591         return index;
1592     }
1593 }
1594
1595 using CamBam.CAD;
1596 using CamBam.Geom;
1597 using System.Collections.Generic;
1598
1599 namespace MorphMuse.Services
1600 {
1601     internal static class LayerGenerator
1602     {
1603         public static List<Polyline> GenerateParallelClosedPolylines(
1604             Polyline closedBase,
1605             List<Point3F> openReferencePoints)
1606         {
1607             var contours = new List<Polyline>();
1608
1609             foreach (Point3F refPt in openReferencePoints)
1610             {
1611                 float offsetValue = (float)refPt.X;
1612                 float zHeight = (float)refPt.Y;
1613
1614                 Polyline[] offsetLayers =
closedBase.CreateOffsetPolyline(offsetValue, 0.01f);
1615                 if (offsetLayers == null || offsetLayers.Length == 0)
1616                 {
1617                     CamBam.ThisApplication.AddLogMessage($"offsetValue: {offsetValue}
// offsetLayers.Length: {(offsetLayers != null ? offsetLayers.Length.ToString() :
"nulo")}");
1618                     continue;
1619                 }
1620
1621                 foreach (Polyline layer in offsetLayers)
1622                 {
1623                     for (int i = 0; i < layer.Points.Count; i++)
1624                     {
1625                         PolylineItem item = layer.Points[i];
1626                         Point3F pt = item.Point;
1627                         pt.Z = zHeight;
1628                         item.Point = pt;
1629                         layer.Points[i] = item;

```



```

1630         }
1631
1632         contours.Add(layer);
1633     }
1634 }
1635
1636     return contours;
1637 }
1638
1639     public static List<List<Polyline>> GenerateContoursByGeratrizOrder(
1640         Polyline closedBase,
1641         List<Point3F> openReferencePoints)
1642     {
1643         var orderedContours = new List<List<Polyline>>();
1644
1645         foreach (var refPt in openReferencePoints)
1646         {
1647             var curves = GenerateParallelClosedPolylines(closedBase, new
List<Point3F> { refPt });
1648             orderedContours.Add(curves);
1649         }
1650
1651         return orderedContours;
1652     }
1653 }
1654 }
1655 using CamBam.CAD;
1656 using CamBam.Geom;
1657 using System;
1658 using System.Collections.Generic;
1659
1660 namespace MorphMuse.Services
1661 {
1662     public static class Geometry3F
1663     {
1664         public static Vector3F FromPoints(Point3F a, Point3F b)
1665         {
1666             return new Vector3F(b.X - a.X, b.Y - a.Y, b.Z - a.Z);
1667         }
1668
1669         public static Vector3F Subtract(Point3F a, Point3F b)
1670         {
1671             return new Vector3F(a.X - b.X, a.Y - b.Y, a.Z - b.Z);
1672         }
1673
1674         public static Vector3F Cross(Vector3F a, Vector3F b)
1675         {
1676             return new Vector3F(
1677                 a.Y * b.Z - a.Z * b.Y,
1678                 a.Z * b.X - a.X * b.Z,
1679                 a.X * b.Y - a.Y * b.X
1680             );
1681         }
1682
1683         public static double Length(Vector3F v)
1684         {
1685             return Math.Sqrt(v.X * v.X + v.Y * v.Y + v.Z * v.Z);
1686         }
1687
1688         public static float Distance(Point3F a, Point3F b)
1689         {
1690             Vector3F delta = FromPoints(a, b);
1691             return (float)Length(delta);
1692         }
1693
1694         public static Point3F GetCentroid(List<Point3F> points)
1695         {
1696             if (points == null || points.Count == 0)
1697                 return new Point3F(0, 0, 0);
1698
1699             float sumX = 0, sumY = 0, sumZ = 0;
1700
1701             foreach (Point3F pt in points)
1702             {

```

```

1702         sumX += (float)pt.X;
1703         sumY += (float)pt.Y;
1704         sumZ += (float)pt.Z;
1705     }
1706
1707     int count = points.Count;
1708     return new Point3F(sumX / count, sumY / count, sumZ / count);
1709 }
1710 public static double AngleBetween(Vector3F u, Vector3F v)
1711 {
1712     double dot = u.X * v.X + u.Y * v.Y + u.Z * v.Z;
1713     double lenU = Geometry3F.Length(u);
1714     double lenV = Geometry3F.Length(v);
1715
1716     if (lenU < 1e-8 || lenV < 1e-8)
1717         return 0.0; // Vetores nulos ou quase nulos
1718
1719     double cosTheta = dot / (lenU * lenV);
1720     cosTheta = Math.Max(-1.0, Math.Min(1.0, cosTheta)); // Clamping para
evitar erros numéricos
1721
1722     return Math.Acos(cosTheta); // Retorna em radianos
1723 }
1724 public static int AddPoint(Point3F p, Point3FArray points,
Dictionary<Point3F, int> indexMap)
1725 {
1726     int index;
1727     if (!indexMap.TryGetValue(p, out index))
1728     {
1729         index = points.Count;
1730         points.Add(p);
1731         indexMap[p] = index;
1732     }
1733     return index;
1734 }
1735 }
1736 }
1737 using CamBam.CAD;
1738 using CamBam.Geom;
1739 using CamBam.UI;
1740 using MorphMuse;
1741 using MorphMuse.Services;
1742 using PluginSettings;
1743 using System;
1744 using System.Collections.Generic;
1745 using System.Drawing;
1746 using System.Windows.Forms;
1747 using System.Linq;
1748
1749 public class MorphMuseController
1750 {
1751     private readonly CamBamUI _ui;
1752     private readonly SettingsManager _settingsManager;
1753
1754     public MorphMuseController(CamBamUI ui)
1755     {
1756         _ui = ui;
1757         _settingsManager = new SettingsManager();
1758     }
1759
1760     public void Execute()
1761     {
1762         if (!PolylineManager.ValidateSelection(out PolylineManager selectionManager))
1763         {
1764             return; // Mensagem de erro já exibida no ValidateSelection
1765         }
1766
1767         // Determina o modo de operação
1768         if (selectionManager.CounterClosedP == 1 && selectionManager.CounterOpenP ==
1)
1769         {
1770             ExecuteVolumeWithCap(selectionManager);
1771         }

```

```

1772         else if (selectionManager.CounterClosedP == 0 &&
selectionManager.CounterOpenP == 2)
1773         {
1774             ExecuteSweepBetweenOpenCurves(selectionManager);
1775         }
1776     }
1777
1778     private void ExecuteVolumeWithCap(PolylineManager selectionManager)
1779     {
1780         var simplifiedClosedCurves = PrepareClosedCurves(selectionManager);
1781         if (simplifiedClosedCurves.Count < 2) return;
1782
1783         string originalLayerName = _ui.ActiveView.CADFile.ActiveLayerName;
1784         string surfaceLayerName = CreateUniqueLayer("MorphVolume");
1785
1786         Point3FArray finalSurfacePoints = new Point3FArray();
1787         Dictionary<Point3F, int> finalPointIndex = new Dictionary<Point3F, int>();
1788         List<TriangleFace> finalSurfaceFaces = new List<TriangleFace>();
1789
1790         // Gera a superf cie lateral (curvas fechadas)
1791         SurfaceBuilderCopilot.GenerateLateralSurface(simplifiedClosedCurves,
finalSurfacePoints, finalPointIndex, finalSurfaceFaces, isClosed: true);
1792
1793         // Gera a tampa (cap)
1794         List<Point3F> topmostSimplifiedCurve =
simplifiedClosedCurves[simplifiedClosedCurves.Count - 1];
1795         SurfaceBuilderCopilot.GenerateCapSurface(topmostSimplifiedCurve,
finalSurfacePoints, finalPointIndex, finalSurfaceFaces);
1796
1797         AddSurfaceToCAD(finalSurfacePoints, finalSurfaceFaces, surfaceLayerName,
originalLayerName);
1798     }
1799
1800     private void ExecuteSweepBetweenOpenCurves(PolylineManager selectionManager)
1801     {
1802         var openPolys = selectionManager.SelectedOpenPolys;
1803         if (openPolys.Count < 2) return;
1804
1805         // Exibe o di logo para o usu rio escolher qual   o RAIL e qual   o
PROFILE
1806         Polyline railPoly;
1807         Polyline profilePoly;
1808
1809         using (var form = new SweepSelectionForm(openPolys))
1810         {
1811             if (form.ShowDialog() != DialogResult.OK) return;
1812             railPoly = form.Rail;
1813             profilePoly = form.Profile;
1814         }
1815
1816         string originalLayerName = _ui.ActiveView.CADFile.ActiveLayerName;
1817         string surfaceLayerName = CreateUniqueLayer("MorphSweep");
1818
1819         Point3FArray finalSurfacePoints = new Point3FArray();
1820         Dictionary<Point3F, int> finalPointIndex = new Dictionary<Point3F, int>();
1821         List<TriangleFace> finalSurfaceFaces = new List<TriangleFace>();
1822
1823         // Prepara as curvas (amostragem e simplifica  o)
1824         var units = SettingsManager.GetUnits();
1825         var adaptiveParams = _settingsManager.GetDefaultAdaptiveParameters();
1826         double dpTolerance =
SettingsManager.ConvertFromMillimeters(adaptiveParams.DouglasPeuckerTolerance, units);
1827
1828         // Converter Rail para List<Point3F>
1829         var railPoints = new List<Point3F>();
1830         for (int i = 0; i < railPoly.Points.Count; i++)
1831         {
1832             var p = railPoly.Points[i];
1833             railPoints.Add(new Point3F((float)p.Point.X, (float)p.Point.Y,
(float)p.Point.Z));
1834         }
1835         var simplifiedRail = PolylineSimplifier.SimplifyDouglasPeucker(railPoints,
dpTolerance);

```

```

1836
1837 // Converter Profile para List<Point3F>
1838 var profilePoints = new List<Point3F>();
1839 for (int i = 0; i < profilePoly.Points.Count; i++)
1840 {
1841     var p = profilePoly.Points[i];
1842     profilePoints.Add(new Point3F((float)p.Point.X, (float)p.Point.Y,
(float)p.Point.Z));
1843 }
1844 var simplifiedProfile =
PolylineSimplifier.SimplifyDouglasPeucker(profilePoints, dpTolerance);
1845
1846 // Gera os contornos de Sweep (posicionando e rotacionando o profile ao longo
do rail)
1847 var sweepContours = SweepGenerator.GenerateSweepContours(simplifiedRail,
simplifiedProfile);
1848
1849 // Gera a superfície lateral entre os contornos gerados (isClosed: false)
1850 SurfaceBuilderCopilot.GenerateLateralSurface(sweepContours,
finalSurfacePoints, finalPointIndex, finalSurfaceFaces, isClosed: false);
1851
1852 AddSurfaceToCAD(finalSurfacePoints, finalSurfaceFaces, surfaceLayerName,
originalLayerName);
1853 }
1854
1855 private void AddSurfaceToCAD(Point3FArray points, List<TriangleFace> faces,
string layerName, string originalLayerName)
1856 {
1857     Surface surfaceEntity = new Surface
1858     {
1859         Points = points,
1860         Faces = faces.ToArray()
1861     };
1862
1863     _ui.ActiveView.CADFile.Layers[layerName].Color = Color.DeepSkyBlue;
1864     _ui.ActiveView.CADFile.Add(surfaceEntity);
1865     _ui.ActiveView.CADFile.SetActiveLayer(originalLayerName);
1866     _ui.ActiveView.ZoomToFit();
1867     _ui.ActiveView.RefreshView();
1868 }
1869
1870 private List<List<Point3F>> PrepareClosedCurves(PolylineManager selectionManager)
1871 {
1872     var units = SettingsManager.GetUnits();
1873     Polyline guideCurve = selectionManager.ClosedPoly;
1874     var adaptiveParams = _settingsManager.GetSmartAdaptiveParameters(guideCurve);
1875
1876     double dpTolerance =
SettingsManager.ConvertFromMillimeters(adaptiveParams.DouglasPeuckerTolerance, units);
1877     double samplingStep =
SettingsManager.ConvertFromMillimeters(adaptiveParams.SamplingStepClosedPoly, units)
/ 5;
1878
1879     var openCurveProcessor = new OpenPolylineProcessor(selectionManager.OpenPoly,
samplingStep, dpTolerance);
1880     var orderedClosedCurves =
LayerGenerator.GenerateContoursByGeratrizOrder(selectionManager.ClosedPoly,
openCurveProcessor.SimplifiedPoints);
1881     var sampledClosedCurves =
CurveSampler.GenerateSampledPointsFromContours(orderedClosedCurves,
openCurveProcessor.SimplifiedPoints, samplingStep, dpTolerance);
1882
1883     return SimplifyAll(sampledClosedCurves, dpTolerance);
1884 }
1885
1886 private List<List<Point3F>> SimplifyAll(List<List<Point3F>> curves, double
tolerance)
1887 {
1888     var result = new List<List<Point3F>>();
1889     foreach (var curve in curves)
1890         result.Add(PolylineSimplifier.SimplifyDouglasPeucker(curve, tolerance));
1891     return result;
1892 }

```

```

1893
1894     private string CreateUniqueLayer(string baseName)
1895     {
1896         int index = 1;
1897         string layerName;
1898         var cadFile = _ui.ActiveView.CADFile;
1899         do
1900         {
1901             layerName = $"{baseName}{index:D3}";
1902             index++;
1903         }
1904         while (cadFile.HasLayer(layerName));
1905
1906         cadFile.CreateLayer(layerName);
1907         cadFile.SetActiveLayer(layerName);
1908         return layerName;
1909     }
1910 }
1911 using CamBam.CAD;
1912 using CamBam.Geom;
1913 using System.Collections.Generic;
1914
1915 namespace MorphMuse.Services
1916 {
1917     internal class OpenPolylineProcessor
1918     {
1919         public List<Point3F> SimplifiedPoints { get; private set; }
1920
1921         public OpenPolylineProcessor(Polyline openPolyline, double arcTolerance,
double simplifyTolerance)
1922         {
1923             //CheckOpenPolylineStart(openPolyline);
1924             SimplifiedPoints = new List<Point3F>();
1925
1926             if (openPolyline != null && !openPolyline.Closed &&
openPolyline.Points.Count > 0)
1927             {
1928                 // 1. Arcs removal
1929                 Polyline linearized = openPolyline.RemoveArcs(arcTolerance);
1930
1931                 // 2. Define origin as first point
1932                 Point3F origin = linearized.Points[0].Point;
1933
1934                 // 3. Extract and translate points to origin
1935                 List<Point3F> translatedPoints = new List<Point3F>();
1936                 foreach (PolylineItem item in linearized.Points)
1937                 {
1938                     Point3F pt = item.Point;
1939                     Point3F relative = new Point3F(
1940                         pt.X - origin.X,
1941                         pt.Y - origin.Y,
1942                         pt.Z - origin.Z
1943                     );
1944                     translatedPoints.Add(relative);
1945                 }
1946
1947                 // 4. Apply Douglas-Peucker simplification
1948                 SimplifiedPoints =
PolylineSimplifier.SimplifyDouglasPeucker(translatedPoints, simplifyTolerance);
1949             }
1950         }
1951     }
1952 }
1953 using CamBam.CAD;
1954 using CamBam.Geom;
1955 using CamBam.UI;
1956 using CamBam.Util;
1957 using System;
1958 using System.Collections.Generic;
1959 using System.Drawing;
1960 using System.Windows.Forms;
1961
1962 namespace MorphMuse.Services

```

```

1963 {
1964     internal class PolylineManager
1965     {
1966         public Polyline ClosedPoly { get; private set; }
1967         public Polyline OpenPoly { get; private set; }
1968         public List<Polyline> SelectedOpenPolys { get; private set; } // Nova lista
para múltiplas polilinhas abertas
1969         public int CounterOpenP { get; private set; }
1970         public int CounterClosedP { get; private set; }
1971
1972         public PolylineManager(Polyline closed, Polyline open)
1973         {
1974             ClosedPoly = closed;
1975             OpenPoly = open;
1976             SelectedOpenPolys = new List<Polyline>();
1977             if (open != null) SelectedOpenPolys.Add(open);
1978         }
1979
1980         public PolylineManager(List<Polyline> openPolys)
1981         {
1982             SelectedOpenPolys = openPolys;
1983             CounterOpenP = openPolys.Count;
1984             CounterClosedP = 0;
1985         }
1986
1987         public static bool TryCreateFromSelection(out PolylineManager manager)
1988         {
1989             manager = null;
1990
1991             GetPolylinesFromSelection(out List<Polyline> closedPolys, out
List<Polyline> openPolys);
1992
1993             int closedCount = closedPolys.Count;
1994             int openCount = openPolys.Count;
1995
1996             // Cenário A: 1 Fechada + 1 Aberta (Volume com tampa)
1997             if (closedCount == 1 && openCount == 1)
1998             {
1999                 Polyline closed = closedPolys[0];
2000                 Polyline open = openPolys[0];
2001
2002                 manager = new PolylineManager(closed, open)
2003                 {
2004                     CounterClosedP = closedCount,
2005                     CounterOpenP = openCount
2006                 };
2007
2008                 float MaxOpenPolyAmplitude = GetOpenPolyEffectiveAmplitudeX(open);
2009                 float MaxNegativeOffset =
FindMaxSafeNegativeOffsetBinarySearch(closed);
2010
2011                 if (MaxOpenPolyAmplitude < MaxNegativeOffset)
2012                 {
2013                     MessageBox.Show(TextTranslation.Translate(
2014                         $"Warning: The open polyline's effective amplitude along the
X axis\nexceeds the maximum negative offset of the closed polyline
({MaxNegativeOffset:F4}).\nThis plugin can't deal with this yet."));
2015                     return false;
2016                 }
2017
2018                 return true;
2019             }
2020             // Cenário B: 2 Abertas (Superfície lateral apenas)
2021             else if (closedCount == 0 && openCount == 2)
2022             {
2023                 manager = new PolylineManager(openPolys);
2024                 return true;
2025             }
2026             else
2027             {
2028                 MessageBox.Show(TextTranslation.Translate(
2029                     "Invalid Selection. Please select either:\n1. One closed and one
open polyline (for volume with cap)\n2. Two open polylines (for surface only)",

```

```

2030         "Invalid Selection",
2031         MessageBoxButtons.OK,
2032         MessageBoxIcon.Warning);
2033     }
2034
2035     return false;
2036 }
2037
2038 public static void GetPolylinesFromSelection(out List<Polyline> closedPolys,
out List<Polyline> openPolys)
2039 {
2040     closedPolys = new List<Polyline>();
2041     openPolys = new List<Polyline>();
2042
2043     ICADView view = CamBamUI.MainUI.ActiveView;
2044
2045     foreach (object entObj in view.SelectedEntities)
2046     {
2047         Entity ent = entObj as Entity;
2048         if (ent == null) continue;
2049
2050         Entity clone = (Entity)ent.Clone();
2051
2052         switch (clone)
2053         {
2054             case Polyline poly:
2055                 if (poly.CanConvertToPolylines == true)
2056                     poly = poly.ConvertToPolylines(true)[0];
2057                 if (poly.Closed) closedPolys.Add(poly);
2058                 else openPolys.Add(poly);
2059                 break;
2060
2061             case Circle circle:
2062                 var polyCircle = circle.ToPolyline();
2063                 if (polyCircle != null) closedPolys.Add(polyCircle);
2064                 break;
2065
2066             case Arc arc:
2067                 var polyArc = arc.ToPolyline();
2068                 if (polyArc != null) openPolys.Add(polyArc);
2069                 break;
2070
2071             case Line line:
2072                 var polyLine = line.ToPolyline();
2073                 if (polyLine != null) openPolys.Add(polyLine);
2074                 break;
2075
2076             case Spline spline:
2077                 var polySpline = spline.ToPolyline(0.01f);
2078                 if (polySpline.Closed) closedPolys.Add(polySpline);
2079                 else openPolys.Add(polySpline);
2080                 break;
2081         }
2082     }
2083 }
2084
2085 public static bool ValidateSelection(out PolylineManager selectionManager)
2086 {
2087     return TryCreateFromSelection(out selectionManager);
2088 }
2089
2090 public static float FindMaxSafeNegativeOffsetBinarySearch(Polyline
closedBase, float tolerance = 0.01f)
2091 {
2092     SizeF amplitude = GetAmplitudeXY(closedBase);
2093     float minOffset = -Math.Min(amplitude.Width, amplitude.Height);
2094     float maxOffset = 0f;
2095     float safeOffset = 0f;
2096
2097     while (Math.Abs(maxOffset - minOffset) > tolerance)
2098     {
2099         float mid = (minOffset + maxOffset) / 2f;
2100         Polyline[] offsetResult = closedBase.CreateOffsetPolyline(mid, 0.01f);

```

```

2101
2102         bool isValid = offsetResult != null &&
2103                     offsetResult.Length == 1 &&
2104                     offsetResult[0].Points.Count >= 3;
2105
2106         if (isValid)
2107         {
2108             safeOffset = mid;
2109             maxOffset = mid;
2110         }
2111         else
2112         {
2113             minOffset = mid;
2114         }
2115     }
2116     return safeOffset;
2117 }
2118
2119 public static SizeF GetAmplitudeXY(Polyline polyline)
2120 {
2121     PointF min = new PointF();
2122     PointF max = new PointF();
2123     polyline.GetExtents(ref min, ref max);
2124     return new SizeF(max.X - min.X, max.Y - min.Y);
2125 }
2126
2127 public static float GetOpenPolyEffectiveAmplitudeX(Polyline poly)
2128 {
2129     if (poly == null || poly.Points.Count < 2) return 0;
2130     float xStart = (float)poly.Points[0].Point.X;
2131     float xEnd = (float)poly.Points[poly.Points.Count - 1].Point.X;
2132     return (xEnd - xStart);
2133 }
2134 }
2135
2136 using CamBam.Geom;
2137 using System.Collections.Generic;
2138 using MorphMuse.Services;
2139
2140 internal static class PolylineSimplifier
2141 {
2142     public static List<Point3F> SimplifyDouglasPeucker(List<Point3F> points, double
2143 tolerance, bool closed = false)
2144     {
2145         if (points == null || points.Count < 3)
2146             return new List<Point3F>(points);
2147
2148         var keep = new bool[points.Count];
2149         keep[0] = true;
2150         keep[points.Count - 1] = true;
2151
2152         Simplify(points, 0, points.Count - 1, tolerance, keep);
2153
2154         if (closed)
2155         {
2156             keep[0] = true;
2157             keep[points.Count - 1] = false; // Remove duplicate for closed polyline
2158         }
2159
2160         var result = new List<Point3F>();
2161         for (int i = 0; i < points.Count; i++)
2162         {
2163             if (keep[i])
2164                 result.Add(points[i]);
2165         }
2166         return result;
2167     }
2168
2169     private static void Simplify(List<Point3F> pts, int start, int end, double tol,
2170 bool[] keep)
2171     {
2172         if (end <= start + 1)
2173             return;

```



```

2172         var a = pts[start];
2173         var b = pts[end];
2174
2175
2176         double maxDist = 0;
2177         int index = -1;
2178
2179         for (int i = start + 1; i < end; i++)
2180         {
2181             double dist = PerpendicularDistance(pts[i], a, b);
2182             if (dist > tol && dist > maxDist)
2183             {
2184                 maxDist = dist;
2185                 index = i;
2186             }
2187         }
2188
2189         if (index != -1)
2190         {
2191             keep[index] = true;
2192             Simplify(pts, start, index, tol, keep);
2193             Simplify(pts, index, end, tol, keep);
2194         }
2195     }
2196
2197     private static double PerpendicularDistance(Point3F p, Point3F a, Point3F b)
2198     {
2199         var ab = Geometry3F.Subtract(b, a);
2200         var ap = Geometry3F.Subtract(p, a);
2201         var cross = Geometry3F.Cross(ab, ap);
2202         double area = Geometry3F.Length(cross);
2203         double baseLength = Geometry3F.Length(ab);
2204         return baseLength == 0 ? 0 : area / baseLength;
2205     }
2206 }
2207 using System;
2208 using System.Text;
2209
2210 public class ProjectLogger
2211 {
2212     private readonly StringBuilder _logBuilder = new StringBuilder();
2213
2214     public void Log(string message)
2215     {
2216         _logBuilder.AppendLine($"{DateTime.Now:yyyy-MM-dd HH:mm:ss} - {message}");
2217     }
2218
2219     public void LogError(string message)
2220     {
2221         _logBuilder.AppendLine($"{DateTime.Now:yyyy-MM-dd HH:mm:ss} [ERROR] -
2222 {message}");
2223     }
2224
2225     public void LogInfo(string message)
2226     {
2227         _logBuilder.AppendLine($"{DateTime.Now:yyyy-MM-dd HH:mm:ss} [INFO] -
2228 {message}");
2229     }
2230
2231     public string GetLog()
2232     {
2233         return _logBuilder.ToString();
2234     }
2235
2236     public void Clear()
2237     {
2238         _logBuilder.Clear();
2239     }
2240 }
2241 using CamBam;
2242 using CamBam.CAD;
2243 using CamBam.Geom;
2244 using CamBam.UI;

```

```

2243 using MorphMuse.Services;
2244 using System;
2245 using System.Collections.Generic;
2246
2247 namespace PluginSettings
2248 {
2249     public class SettingsManager
2250     {
2251         public enum Units
2252         {
2253             Millimeters = 0,
2254             Inches = 1,
2255             Centimeters = 2,
2256             Meters = 3,
2257             Thousandths = 4,
2258             Unknown = -1
2259         }
2260
2261         public Units CurrentUnits { get; private set; } = Units.Millimeters;
2262
2263         // Default values stored in millimeters
2264         private readonly double defaultDouglasPeuckerTolerance = 0.001;
2265         private readonly double defaultSamplingStep = 0.005;
2266
2267         // Get current units from CamBam
2268         public static Units GetUnits()
2269         {
2270             try
2271             {
2272                 return (Units)CamBamUI.MainUI.ActiveView.CADFile.DrawingUnits;
2273             }
2274             catch
2275             {
2276                 return Units.Unknown;
2277             }
2278         }
2279
2280         public class AdaptiveParameters
2281         {
2282             public double DouglasPeuckerTolerance { get; set; }
2283             public double SamplingStepClosedPoly { get; set; }
2284         }
2285
2286         // Calculates the diagonal of the bounding box for the guide curve using
2287         Polyline.GetExtrema
2288         public double BoundingBoxDiagonal(Polyline guideCurve)
2289         {
2290             Point3F min = new Point3F();
2291             Point3F max = new Point3F();
2292
2293             // Use Polyline's GetExtrema method to find the bounding box corners
2294             guideCurve.GetExtrema(ref min, ref max);
2295
2296             // Calculate the diagonal distance between min and max points
2297             double diagonal = Point3F.Distance(min, max);
2298             return diagonal; // drawing units
2299         }
2300
2301         public AdaptiveParameters GetSmartAdaptiveParameters(Polyline guideCurve)
2302         {
2303             double diagonal = BoundingBoxDiagonal(guideCurve); // units of the drawing
2304             double diagonalMm = ConvertToMillimeters(diagonal, GetUnits());
2305             double stepResolution = CamBamConfig.Defaults.STEPResolution;
2306
2307             // Normalized scale from 0.0 to 1.0
2308             // Values above this will use the maximum tolerances
2309             double scale = Math.Min(1.0, diagonalMm / 1000.0);
2310
2311             // Between these ranges
2312             double dpMin = 0.01;
2313             double dpMax = 0.3;
2314             double stepMin = 0.2;
2315             double stepMax = 2.0;

```

```

2315
2316         double dpTolerance = Lerp(dpMin, dpMax, scale);
2317         double samplingStep = Lerp(stepMin, stepMax, scale);
2318
2319         // Clamping with cambam STEPResolution
2320         //dpTolerance = Clamp(dpTolerance, stepResolution / 10, stepResolution);
2321         //samplingStep = Clamp(samplingStep, stepResolution, 10 * stepResolution);
2322         dpTolerance = Clamp(dpTolerance, stepResolution / 50, stepResolution /
10);
2323         samplingStep = Clamp(samplingStep, 0.05, 2 * stepResolution);
2324
2325         return new AdaptiveParameters
2326         {
2327             DouglasPeuckerTolerance = dpTolerance,
2328             SamplingStepClosedPoly = samplingStep
2329         };
2330     }
2331
2332     // Interpolates linearly between min and max based on t (0.0 to 1.0)
2333     private double Lerp(double min, double max, double t)
2334     {
2335         return min + (max - min) * t;
2336     }
2337     // Calculates tolerance and step adaptively based on the object size
2338
2339     // Utility function to clamp values
2340     public static double Clamp(double value, double min, double max)
2341     {
2342         if (value < min) return min;
2343         if (value > max) return max;
2344         return value;
2345     }
2346
2347     public static double ConvertFromMillimeters(double valueInMm, Units
targetUnits)
2348     {
2349         switch (targetUnits)
2350         {
2351             case Units.Inches:
2352                 return valueInMm / 25.4;
2353             case Units.Centimeters:
2354                 return valueInMm / 10.0;
2355             case Units.Meters:
2356                 return valueInMm / 1000.0;
2357             default: // Millimeters or unknown
2358                 return valueInMm;
2359         }
2360     }
2361
2362     public static double ConvertToMillimeters(double value, Units sourceUnits)
2363     {
2364         switch (sourceUnits)
2365         {
2366             case Units.Inches:
2367                 return value * 25.4;
2368             case Units.Centimeters:
2369                 return value * 10.0;
2370             case Units.Meters:
2371                 return value * 1000.0;
2372             default:
2373                 return value; // already in mm
2374         }
2375     }
2376
2377     public AdaptiveParameters GetDefaultAdaptiveParameters()
2378     {
2379         return new AdaptiveParameters
2380         {
2381             DouglasPeuckerTolerance = defaultDouglasPeuckerTolerance,
2382             SamplingStepClosedPoly = defaultSamplingStep
2383         };
2384     }
2385 }

```

```

2386 }
2387 using PluginSettings;
2388 using System;
2389
2390 namespace MorphMuse.Services
2391 {
2392     // Pure data class for configuration
2393     public class SettingsProfile
2394     {
2395         public string Name { get; set; } = "default";
2396         public SettingsManager.Units Units { get; set; } =
SettingsManager.Units.Millimeters;
2397         public float Tolerance { get; set; } = 0.01f;
2398         public float DouglasPeuckerTolerance { get; set; } = 0.001f;
2399         public float SamplingStepClosedPoly { get; set; } = 0.05f;
2400         public float OffsetStep { get; set; } = 0.5f;
2401         public float BaseDensity { get; set; } = 0.5f;
2402         public float MinimumSamplingStep { get; set; } = 0.1f;
2403         public float DegeneracyThreshold { get; set; } = 1e-6f;
2404         public float VectorLengthThreshold { get; set; } = 1e-8f;
2405         public int MaxLogPoints { get; set; } = 5;
2406         public int MinimumCurveLengthForAlignment { get; set; } = 3;
2407         public bool UseAdvancedMode { get; set; } = false;
2408     }
2409 }
2410 using CamBam.CAD;
2411 using CamBam.Geom;
2412 using System;
2413 using System.Collections.Generic;
2414 using System.Linq;
2415
2416 namespace MorphMuse.Services
2417 {
2418     public static class SurfaceBuilderCopilot
2419     {
2420         /// <summary>
2421         /// Constrói uma superfície entre duas curvas (abertas ou fechadas).
2422         /// </summary>
2423         public static Surface BuildSurfaceBetweenCurves(List<Point3F> lower,
List<Point3F> upper, bool isClosed = true)
2424         {
2425             var points = new Point3FArray();
2426             var pointIndex = new Dictionary<Point3F, int>();
2427             var faces = new List<TriangleFace>();
2428
2429             // Sincroniza a rota da curva superior com a inferior (apenas se
forem fechadas)
2430             if (isClosed)
2431             {
2432                 int matchIndex = FindClosestIndex(lower[0], upper);
2433                 upper = RotateCurve(upper, matchIndex);
2434
2435                 // Fecha as curvas se não forem fechadas (e deveriam ser)
2436                 if (Geometry3F.Distance(lower[0], lower[lower.Count - 1]) > 1e-6)
2437                     lower.Add(lower[0]);
2438
2439                 if (Geometry3F.Distance(upper[0], upper[upper.Count - 1]) > 1e-6)
2440                     upper.Add(upper[0]);
2441             }
2442
2443             int i = 0, j = 0;
2444
2445             // Triangulação adaptativa baseada em distância
2446             while (i < lower.Count - 1 && j < upper.Count - 1)
2447             {
2448                 Point3F a = lower[i];
2449                 Point3F b = upper[j];
2450                 Point3F aNext = lower[i + 1];
2451                 Point3F bNext = upper[j + 1];
2452
2453                 double dANextToB = Geometry3F.Distance(aNext, b);
2454                 double dAtoBNext = Geometry3F.Distance(a, bNext);
2455

```

```

2456         if (dANextToB < dAtoBNext)
2457         {
2458             AddTriangle(a, b, aNext, points, pointIndex, faces);
2459             i++;
2460         }
2461         else
2462         {
2463             AddTriangle(a, b, bNext, points, pointIndex, faces);
2464             j++;
2465         }
2466     }
2467
2468     // Consume os pontos restantes
2469     while (i < lower.Count - 1)
2470     {
2471         AddTriangle(lower[i], upper[j], lower[i + 1], points, pointIndex,
faces);
2472         i++;
2473     }
2474
2475     while (j < upper.Count - 1)
2476     {
2477         AddTriangle(lower[i], upper[j], upper[j + 1], points, pointIndex,
faces);
2478         j++;
2479     }
2480
2481     return new Surface
2482     {
2483         Points = points,
2484         Faces = faces.ToArray()
2485     };
2486 }
2487
2488     /// <summary>
2489     /// Gera a superfície de fechamento (cap) usando o algoritmo de Ear Clipping
para suportar curvas c ncavas.
2490     /// </summary>
2491     public static void GenerateCapSurface(List<Point3F> topCurve, Point3FArray
points, Dictionary<Point3F, int> indexMap, List<TriangleFace> faces)
2492     {
2493         // Substitua mos o Triangle Fan pelo Ear Clipping para suportar
pol gonos c ncavos
2494         var capFaces = EarClippingTriangulator.Triangulate(topCurve, points,
indexMap);
2495         faces.AddRange(capFaces);
2496     }
2497
2498     /// <summary>
2499     /// Gera a superfície lateral entre m ltiplas curvas.
2500     /// </summary>
2501     public static void GenerateLateralSurface(List<List<Point3F>>
simplifiedCurves, Point3FArray points, Dictionary<Point3F, int> indexMap,
List<TriangleFace> faces, bool isClosed = true)
2502     {
2503         for (int i = 0; i < simplifiedCurves.Count - 1; i++)
2504         {
2505             var lower = simplifiedCurves[i];
2506             var upper = simplifiedCurves[i + 1];
2507
2508             if (isClosed)
2509             {
2510                 AlignCurveToPrevious(lower, upper); // Alinha apenas se forem
fechadas
2511             }
2512
2513             Surface partialSurface = BuildSurfaceBetweenCurves(lower, upper,
isClosed);
2514
2515             foreach (var face in partialSurface.Faces)
2516             {
2517                 Point3F pa = partialSurface.Points[face.A];
2518                 Point3F pb = partialSurface.Points[face.B];

```

```

2519         Point3F pc = partialSurface.Points[face.C];
2520
2521         int ia = Geometry3F.AddPoint(pa, points, indexMap);
2522         int ib = Geometry3F.AddPoint(pb, points, indexMap);
2523         int ic = Geometry3F.AddPoint(pc, points, indexMap);
2524
2525         faces.Add(new TriangleFace(ia, ib, ic));
2526     }
2527 }
2528
2529 private static void AddTriangle(Point3F a, Point3F b, Point3F c,
2530     Point3FArray points,
2531     Dictionary<Point3F, int> pointIndex,
2532     List<TriangleFace> faces)
2533 {
2534     if (IsDegenerate(a, b, c)) return;
2535
2536     int ia = Geometry3F.AddPoint(a, points, pointIndex);
2537     int ib = Geometry3F.AddPoint(b, points, pointIndex);
2538     int ic = Geometry3F.AddPoint(c, points, pointIndex);
2539
2540     // Orientação fixa: sempre a â' c â' b
2541     faces.Add(new TriangleFace(ia, ic, ib));
2542 }
2543
2544 private static bool IsDegenerate(Point3F a, Point3F b, Point3F c)
2545 {
2546     var ab = Geometry3F.FromPoints(a, b);
2547     var ac = Geometry3F.FromPoints(a, c);
2548     var cross = Geometry3F.Cross(ab, ac);
2549     return Geometry3F.Length(cross) * 0.5 < 1e-6;
2550 }
2551
2552 public static void AlignCurveToPrevious(List<Point3F> previous, List<Point3F>
2553 current)
2554 {
2555     int n = current.Count;
2556     int m = previous.Count;
2557     if (n < 3 || m < 3) return;
2558
2559     int minCount = Math.Min(n, m);
2560     double bestScore = double.MaxValue;
2561     int bestOffset = 0;
2562
2563     for (int offset = 0; offset < n; offset++)
2564     {
2565         double score = 0;
2566         for (int i = 0; i < minCount; i++)
2567         {
2568             int j = (i + offset) % n;
2569             score += Geometry3F.Distance(previous[i], current[j]);
2570         }
2571
2572         if (score < bestScore)
2573         {
2574             bestScore = score;
2575             bestOffset = offset;
2576         }
2577     }
2578
2579     if (bestOffset > 0)
2580     {
2581         var rotated = new List<Point3F>();
2582         rotated.AddRange(current.Skip(bestOffset));
2583         rotated.AddRange(current.Take(bestOffset));
2584         current.Clear();
2585         current.AddRange(rotated);
2586     }
2587 }
2588
2589 static int FindClosestIndex(Point3F target, List<Point3F> curve)
2590 {

```

```

2591         int bestIndex = 0;
2592         double bestDistance = Geometry3F.Distance(target, curve[0]);
2593
2594         for (int i = 1; i < curve.Count; i++)
2595         {
2596             double d = Geometry3F.Distance(target, curve[i]);
2597             if (d < bestDistance)
2598             {
2599                 bestDistance = d;
2600                 bestIndex = i;
2601             }
2602         }
2603
2604         return bestIndex;
2605     }
2606
2607     static List<Point3F> RotateCurve(List<Point3F> curve, int offset)
2608     {
2609         var result = new List<Point3F>();
2610         for (int i = 0; i < curve.Count; i++)
2611             result.Add(curve[(i + offset) % curve.Count]);
2612         return result;
2613     }
2614 }
2615
2616 using CamBam.Geom;
2617 using System;
2618 using System.Collections.Generic;
2619
2620 namespace MorphMuse.Services
2621 {
2622     public static class SweepGenerator
2623     {
2624         /// <summary>
2625         /// Gera uma s rie de contornos (se  es transversais) posicionando e
2626         rotacionando a curva 'profile' ao longo da curva 'rail'.
2627         /// </summary>
2628         public static List<List<Point3F>> GenerateSweepContours(List<Point3F> rail,
2629         List<Point3F> profile)
2630         {
2631             var contours = new List<List<Point3F>>();
2632             if (rail.Count < 2 || profile.Count < 2) return contours;
2633
2634             // 1. Normalizar o profile para que o primeiro ponto seja a origem (0,0,0)
2635             // Isso garante que o profile "grude" no trilho pelo seu in cio.
2636             Point3F profileOrigin = profile[0];
2637             var normalizedProfile = new List<Point3F>();
2638             foreach (var p in profile)
2639             {
2640                 normalizedProfile.Add(new Point3F(p.X - profileOrigin.X, p.Y -
2641                 profileOrigin.Y, p.Z - profileOrigin.Z));
2642             }
2643
2644             // 2. Percorrer o trilho e posicionar o profile em cada ponto
2645             for (int i = 0; i < rail.Count; i++)
2646             {
2647                 Point3F currentPos = rail[i];
2648                 Vector3F tangent;
2649
2650                 // Calcular a tangente no ponto atual do trilho
2651                 if (i < rail.Count - 1)
2652                 {
2653                     tangent = Geometry3F.FromPoints(rail[i], rail[i + 1]);
2654                 }
2655                 else
2656                 {
2657                     tangent = Geometry3F.FromPoints(rail[i - 1], rail[i]);
2658                 }
2659
2660                 // Normalizar a tangente
2661                 double len = Geometry3F.Length(tangent);
2662                 if (len > 1e-9)
2663                 {

```

```

2661         tangent = new Vector3F(tangent.X / len, tangent.Y / len,
tangent.Z / len);
2662     }
2663     else
2664     {
2665         tangent = new Vector3F(0, 0, 1); // Fallback
2666     }
2667
2668     // Criar a matriz de transformaçã$ão para alinhar o profile com a
tangente
2669     // Usamos um sistema de coordenadas local (Frenet-Serret simplificado)
2670     // Z local = Tangente
2671     // X local = Vetor perpendicular à tangente (ex: projetado no plano
XY)
2672     // Y local = Cross(Z, X)
2673
2674     Vector3F up = new Vector3F(0, 0, 1);
2675     if (Math.Abs(tangent.Z) > 0.9) up = new Vector3F(1, 0, 0); // Evita
singularidade se a tangente for vertical
2676
2677     Vector3F localX = Geometry3F.Cross(up, tangent);
2678     double lenX = Geometry3F.Length(localX);
2679     if (lenX > 1e-9)
2680     {
2681         localX = new Vector3F(localX.X / lenX, localX.Y / lenX, localX.Z
/ lenX);
2682     }
2683     else
2684     {
2685         localX = new Vector3F(1, 0, 0);
2686     }
2687
2688     Vector3F localY = Geometry3F.Cross(tangent, localX);
2689
2690     // Gerar o contorno transformado
2691     var contour = new List<Point3F>();
2692     foreach (var p in normalizedProfile)
2693     {
2694         // Transformaçã$ão: P_world = P_local.x * localX + P_local.y *
localY + P_local.z * tangent + currentPos
2695         double worldX = p.X * localX.X + p.Y * localY.X + p.Z * tangent.X
+ currentPos.X;
2696         double worldY = p.X * localX.Y + p.Y * localY.Y + p.Z * tangent.Y
+ currentPos.Y;
2697         double worldZ = p.X * localX.Z + p.Y * localY.Z + p.Z * tangent.Z
+ currentPos.Z;
2698
2699         contour.Add(new Point3F((float)worldX, (float)worldY,
(float)worldZ));
2700     }
2701     contours.Add(contour);
2702 }
2703
2704     return contours;
2705 }
2706 }
2707 }
2708 using System;
2709 using System.Collections.Generic;
2710 using System.Drawing;
2711 using System.Windows.Forms;
2712 using CamBam.CAD;
2713
2714 namespace MorphMuse.Services
2715 {
2716     public class SweepSelectionForm : Form
2717     {
2718         private List<Polyline> _polys;
2719         private RadioButton _rbRail1;
2720         private RadioButton _rbRail2;
2721         private Button _btnOk;
2722         private Button _btnCancel;
2723

```



```

2724 public Polyline Rail { get; private set; }
2725 public Polyline Profile { get; private set; }
2726
2727 public SweepSelectionForm(List<Polyline> polys)
2728 {
2729     _polys = polys;
2730     InitializeComponent();
2731 }
2732
2733 private void InitializeComponent()
2734 {
2735     this.Text = "MorphMuse - Sweep Selection";
2736     this.Size = new Size(400, 250);
2737     this.FormBorderStyle = FormBorderStyle.FixedDialog;
2738     this.StartPosition = FormStartPosition.CenterParent;
2739     this.MaximizeBox = false;
2740     this.MinimizeBox = false;
2741
2742     Label lblInstruction = new Label()
2743     {
2744         Text = "Select which curve is the RAIL (path):",
2745         Location = new Point(20, 20),
2746         Size = new Size(350, 20),
2747         Font = new Font(this.Font, FontStyle.Bold)
2748     };
2749
2750     // OpÃ§Ã£o 1
2751     string info1 = GetEntityInfo(_polys[0]);
2752     _rbRail1 = new RadioButton()
2753     {
2754         Text = $"Curve 1: {info1}",
2755         Location = new Point(30, 50),
2756         Size = new Size(340, 30),
2757         Checked = true
2758     };
2759
2760     // OpÃ§Ã£o 2
2761     string info2 = GetEntityInfo(_polys[1]);
2762     _rbRail2 = new RadioButton()
2763     {
2764         Text = $"Curve 2: {info2}",
2765         Location = new Point(30, 90),
2766         Size = new Size(340, 30)
2767     };
2768
2769     _btnOk = new Button()
2770     {
2771         Text = "OK",
2772         DialogResult = DialogResult.OK,
2773         Location = new Point(200, 160),
2774         Size = new Size(80, 30)
2775     };
2776     _btnOk.Click += (s, e) => {
2777         if (_rbRail1.Checked) {
2778             Rail = _polys[0];
2779             Profile = _polys[1];
2780         } else {
2781             Rail = _polys[1];
2782             Profile = _polys[0];
2783         }
2784         this.Close();
2785     };
2786
2787     _btnCancel = new Button()
2788     {
2789         Text = "Cancel",
2790         DialogResult = DialogResult.Cancel,
2791         Location = new Point(290, 160),
2792         Size = new Size(80, 30)
2793     };
2794
2795     this.Controls.Add(lblInstruction);
2796     this.Controls.Add( rbRail1);

```

```

2797         this.Controls.Add(_rbRail2);
2798         this.Controls.Add(_btnOk);
2799         this.Controls.Add(_btnCancel);
2800
2801         this.AcceptButton = _btnOk;
2802         this.CancelButton = _btnCancel;
2803     }
2804
2805     private string GetEntityInfo(Polyline poly)
2806     {
2807         // Tenta obter o ID da entidade no desenho (Entity.ID)
2808         // e o tipo (Polyline, Arc, etc)
2809         string type = poly.GetType().Name;
2810         return $"{type} (ID: {poly.ID}) - Points: {poly.Points.Count}";
2811     }
2812 }
2813
2814 using CamBam.UI;
2815 using System;
2816 using System.Windows.Forms;
2817
2818 public static class Program
2819 {
2820     public static CamBamUI _ui;
2821
2822     public static void InitPlugin(CamBamUI ui)
2823     {
2824         _ui = ui;
2825         ToolStripMenuItem mi = new ToolStripMenuItem("MorphMuse");
2826         mi.Click += MorphMuse_Click;
2827         ui.Menus.mnuPlugins.DropDownItems.Add(mi);
2828     }
2829
2830     static void MorphMuse_Click(object sender, EventArgs e)
2831     {
2832         var controller = new MorphMuseController(_ui);
2833         controller.Execute();
2834     }
2835 }
2836

```